

Pi Server 코드 이해 가이드

이 파일은 자동 생성됩니다. 직접 수정하지 말고 C/C++ 소스의 Doxygen 주석 또는 docs/architecture/README.md를 수정한 뒤 문서 빌드를 다시 실행하십시오.

- 생성 기준: 현재 작업 트리
- 분석 파일 수: 192
- 생성 명령: cmake --build cmake-build --target docs

Pi Server Architecture

WiseAI IVA 차량 탐지의 MQTT parsing 및 슬롯 매핑 계약은 `docs/IVA\_VEHICLE\_DETECTION.md`를 참고한다.

기준일: 2026-08-12

기준 코드: chore/cleanup-unused-features_EVDA-205 (7968a58)

이 문서는 외부 발표 자료가 아니라 현재 Pi_Server C/C++ 코드에 실제로 연결된 런타임 아키텍처를 설명한다. 외부 기획서와 발표 자료는 docs/reference/에 보관한다.

1. 시스템 경계

```
flowchart LR
    Hall[홀센서] --> STM32[STM32]
    STM32 -->|UART text / LoRa frame| SensorLink[SensorLinkManager]
    TestMQTT[개발용 Hall MQTT] --> Mosquitto[Mosquitto]
    SensorLink --> HallService[HallParkingService]
    Mosquitto --> MqttBridge[MqttEventBridge]
    MqttBridge --> HallService

    Camera[Hanwha Vision Camera] -->|RTSP video| FrameBuffer[CameraChannel FrameBuffer]
    Camera -->|CV Snapshot OpenAPI / JPEG| CameraApi[CameraSnapshotApiClient]
    Camera -->|ONVIF MQTT event| Mosquitto
    Camera -.->|BESTSHOT_ENABLED=true일 때 RTSP metadata| BestShot[BestShotReceiver]

    HallService --> Session[(SQLite PARKING_SESSION)]
    HallService --> Evidence[EvidenceCaptureWorker]
    HallService --> Scheduler[CaptureSchedulerRuntime]
    Evidence --> FrameBuffer
    Scheduler --> CameraApi
    Scheduler -. API disabled/fallback .-> FrameBuffer
    CameraApi --> Snapshot
    FrameBuffer --> Snapshot[SnapshotStorage ROI crop]
    Snapshot --> Images[(Image files)]
    Snapshot --> ImageLog[(SQLite IMAGE_LOG)]
    Scheduler --> OCR[OcrWorker]
    OCR -->|HTTPS| Gemini[Gemini API]
    OCR --> Vehicle[(SQLite VEHICLE)]
    OCR --> Timer[ParkingSlotManager / TimerManager]

    Timer --> EventManager[EventManager]
    EventManager -->|MQTT event/state| Qt[Qt Client]
    Qt -->|HTTP or HTTPS API| Http[ParkingHttpServer]
    Http --> Session
    Http --> ImageLog
    Http --> Images
```

Raspberry Pi는 카메라 RTSP를 Qt에 중계하지 않는다. Snapshot API 전용 모드에서는 Pi도 RTSP를 수신하지 않고 이벤트 시점에 original/enhanced JPEG만 요청한다. Qt의 실시간 영상 표시는 카메라와 Qt 사이의 직접 RTSP 연결 책임이다.

EVDA-192에서는 CH1의 WiseAI name1~name4를 EV01~EV04에 1:1로 묶고 네이티브 IvaArea 상태를 슬롯별 OR 조건으로 집계한다.

PARKING_OCCUPANCY_SOURCE=HYBRID_OR이면 영역 하나의 INTRUSION 또는 5초 유지된 Hall OCCUPIED 중 먼저 확정된 입력으로 세션 하나를 만들고, 나중 입력은 동일 세션의 확인 상태만 보강한다. IVA만 확인한 세션은 IVA EXIT, Hall만 확인한 세션은 Hall VACANT이 종료 권한을 가지며, 둘 다 확인한 세션은 두 입력이 모두 VACANT일 때 종료한다. IVA EXIT는 기본 20초 확인 후 확정한다. 신규 이미지는 ch1/EV01/<stage>/에 저장하고 파일명과 DB에 session_id를 보존한다. Snapshot API 모드에서는 좌표 확정 전까지 카메라의 전체 original/enhanced 프레임을 저장한다.

2. 프로세스 시작 순서

src/main.cpp가 composition root다. 생성과 시작 순서는 다음과 같다.

```
AppConfig 환경변수 로드
→ CameraChannel 생성
→ SQLite open / runtime migration
→ SystemEventReporter 시작
→ ParkingHttpServer 시작
→ 선택적 RTSP, Snapshot API, Scheduler, Timer, OCR 객체 구성
→ EvidenceCaptureWorker 시작
→ RTSP fallback 모드일 때만 RtspStreamReceiver 시작. 최초 frame 대기
→ OcrWorker 시작
→ `BESTSHOT_ENABLED=true`일 때만 BestShotReceiver 시작
→ MqttEventBridge 연결
→ CaptureSchedulerRuntime 시작
→ UART/LoRa SensorLinkManager 시작
→ 기존 ACTIVE 타이머 복구
→ SIGINT/SIGTERM 대기
```

Snapshot API가 비활성인 경우 RTSP URL이 없거나 최초 프레임을 받지 못하면 실패한다. Snapshot API가 활성화고 fallback이 꺼져 있으면 RTSP 없이 서버를 시작한다.

3. 홀센서 입차 및 OCR 흐름

```
sequenceDiagram
    participant S as STM32 / Test MQTT
    participant H as HallParkingService
    participant DB as EventDatabase
    participant E as EvidenceCaptureWorker
    participant C as CaptureSchedulerRuntime
    participant A as CameraSnapshotApiClient
    participant F as RTSP FrameBuffer
    participant O as OcrWorker / Gemini
    participant T as ParkingSlotManager

    S->>H: SENSOR:HALL01:OCCUPIED:sequence
    H->>H: parse / sensor-slot mapping / sequence check
    H->>H: optional OCCUPIED confirmation gate
    H->>DB: PARKING_SESSION ACTIVE 생성
    DB->>H: SQLite integer session_id
    H->>E: 시작 증거 + T0+overstay 예약
    H->>C: T0+30s / T0+60s 예약
    E->>F: 즉시 최신 ROI frame
    E->>DB: OCCUPANCY_START_EVIDENCE
    C->>A: T0+30s POST /images/generate
    A-->>C: 같은 frame original/enhanced JPEG
    C->>DB: HALL_30S IMAGE_LOG
    C->>O: 30초 이미지 OCR
    alt 30초 OCR 성공
        O->>DB: 번호판 / 차량 분류 반영
        C->>DB: 60초 이미지는 증거로만 저장
    else 30초 OCR 실패
        C->>A: T0+60s POST /images/generate
        A-->>C: 같은 frame original/enhanced JPEG
        C->>DB: HALL_60S IMAGE_LOG
        C->>O: 60초 이미지 OCR 재시도
    end
    end
    O->>DB: VEHICLE EV/PHEV/NON_EV/UNKNOWN 조회
    alt EV or PHEV
        O->>T: 같은 session_id로 타이머 등록
    else NON_EV or UNKNOWN
        O->>T: 경고 이벤트, EV 장기점유 타이머 미등록
    end
    end
```

3.1 입력 transport

입력	활성화 설정	처리 경로
개발용 MQTT	HALL_MQTT_INPUT_ENABLED=true	MqttEventBridge → HallParkingService
UART line	SENSOR_LINK_MODE=uart-line	UartDriver → SensorLinkManager → HallParkingService

LoRa frame	SENSOR_LINK_MODE=lora-frame	UartDriver → LoRaDriver CRC16 → HallParkingService
------------	-----------------------------	--

config/parking_slots.json의 sensor_id가 STM32 메시지의 센서 ID와 일치해야 한다.

3.2 30초·60초 촬영 활성화

```
CAPTURE_SCHED_ENABLED=true
HALL_CAPTURE_OCR_ENABLED=true
CAPTURE_OFFSETS_SEC=30,60
CAPTURE_OCR_MAX_ATTEMPTS=2
CAMERA_SNAPSHOT_API_ENABLED=true
CAMERA_OPEN_API_BASE=http://CAMERA_IP/opensdk/APP_ID
CAMERA_IMAGE_BASE=http://CAMERA_IP:8080
CAMERA_IMAGE_SERVER_PORT=8080
```

CAPTURE_SCHED_ENABLED의 코드 기본값은 false다. 비활성 상태에서는 30/60초 경로 대신 OCCUPANCY_START_EVIDENCE를 OCR 입력으로 사용한다.

3.3 실제 촬영 의미

CAMERA_SNAPSHOT_API_ENABLED=true이면 HallCaptureExecutor와 EvidenceCaptureWorker는 CV5 CAP의 /images/generate를 호출하고 응답에 포함된 같은 run의 original/enhanced JPEG를 즉시 다운로드한다. OcrWorker는 제공된 enhanced 경로를 사용하므로 Pi OpenCV 화질 개선을 실행하지 않는다. API가 비활성이면 기존 CameraChannel::latest_full_frame ROI 촬영을 사용하며, API 실패 시 RTSP fallback은 별도 설정으로만 허용한다.

현재 CV Snapshot API 계약에는 ROI 입력이 없기 때문에 카메라는 채널 전체 프레임을 반환한다. Pi는 실제 촬영 순간 실행 중인 슬롯별 ROI를 조회하고 original/enhanced를 같은 좌표로 crop한 뒤 enhanced crop을 OCR에 전달한다. ROI는 HTTP PUT으로 즉시 변경하며 SQLite SYSTEM_SETTINGS에 영구 저장한다.

4. 조기 출차와 장기 점유

장기 점유 판정과 OVERSTAY_EVIDENCE 촬영은 독립된 환경변수가 아니라 SYSTEM_SETTINGS.overstay_threshold_seconds 단일 값을 사용한다. Qt는 GET/PUT /api/v1/settings/overstay-threshold로 시간·분·초 UI의 총 초 값을 조회·변경한다. 변경 시 TimerManager의 이전 generation은 무효화되고 활성 타이머와 증거 작업은 원래 입차 T0 기준으로 재예약된다.

4.1 1시간 이전 VACANT

```
HallParkingService VACANT
→ PARKING_SESSION 종료 / PARKING_SLOT VACANT
→ CaptureScheduler 30/60초 예약 취소
→ EvidenceCaptureWorker 장기점유 예약 취소
→ 진행 중 OCR 결과 반영 차단
→ TimerManager 취소
→ 위반 전이면 해당 session_id 이미지 파일 삭제
→ IMAGE_LOG 행 삭제
```

4.2 T0+1시간 이상 점유

```
EvidenceCaptureWorker
→ Camera Snapshot API 전체 original/enhanced frame
→ 실행 중 슬롯 ROI 조회 및 동일 영역 crop
→ OVERSTAY_EVIDENCE 파일 / IMAGE_LOG 저장

TimerManager
→ PARKING_SESSION.status = VIOLATION
→ violation_at 기록
→ VIOLATION_TRIGGERED
→ EventManager
→ Qt MQTT OVERTIME_VIOLATION
```

증거 worker와 타이머는 독립 스레드지만, 타이머가 먼저 깨어나 증거 경로가 비어 있으면 증거 작업을 즉시 앞당기고 500ms 간격으로 최대 5초만 기다린다. 이후에도 실패하면 TIMER_ERROR를 남기고 이미지 없이 위반 알림은 계속하여 알림 자체가 유실되지 않게 한다.

서버 재시작 시에는 SQLite의 활성 세션 entry_time을 원래 T0로 사용해 아직 없는 증거만 복원한 후 장기점유 타이머를 복원한다. VACANT 취소는 priority queue에서 해당 세션 Job을 즉시 제거해 용량을 회수한다.

홀센서 DB/파일 작업 큐는 PARKING_HALL_WORK_QUEUE_CAPACITY(기본 100)로 제한한다. 같은 슬롯의 대기 이벤트는 최신 상태로 병합하고, 서로 다른 슬롯의 신규 이벤트가 용량을 넘을 때만 상태 변경 전에 거부하여 HALL_WORK_QUEUE_OVERFLOW를 기록한다.

5. 카메라 이벤트 경로

5.1 IVA MQTT

```
Camera ONVIF MQTT
→ MqttEventBridge
→ CameraEventParser
→ active IVA Area 확인
→ Area name 또는 channel로 EV01~EV04 매핑
→ Snapshot API original/enhanced 다운로드 후 ROI crop
→ scene JPEG + OpenCV enhanced 생성
→ EVENT_LOG / IMAGE_LOG
→ OcrWorker
→ Qt 카메라 이벤트 topic 발행
```

MotionAlarm, MotionDetection, ObjectDetection 등 비 IVA 이벤트는 중복 사진을 막기 위해 현재 Snapshot을 저장하지 않는다.

5.2 선택적 BestShot metadata

```
Camera RTSP metadata track
→ BestShotReceiver
→ ONVIF XML Object / ImageRef 파싱
→ Camera HTTPS Digest 인증 JPEG 다운로드
→ Vehicle BestShot이 별도 PARKING_SESSION 생성
→ Plate BestShot을 같은 채널 세션에 연결
→ Gemini OCR
```

이 경로는 현재 BESTSHOT_ENABLED=false로 비활성화되어 스레드, RTSP metadata 연결, JPEG 다운로드를 시작하지 않는다. 운영 기본 경로는 WiseAI IVA MQTT → Snapshot API → ROI crop → Gemini OCR이다.

6. 저장 구조

6.1 SQLite

테이블	책임
VEHICLE	번호판, EV/PHEV 등록 정보
PARKING_SLOT	슬롯 종류, 점유 상태, 센서 종류
PARKING_SESSION	입차, 출차, 위반, 번호판, 차량 연결
IMAGE_LOG	원본/개선 파일 경로, OCR, 촬영 사유
EVENT_LOG	주차·OCR·오류·알림 이벤트

활성 세션은 슬롯당 하나만 허용하며, 증거 이미지는 session_id + evidence_reason당 하나만 허용한다.

6.2 이미지 파일

```
data/snapshots/
├── ch1/
│   ├── EV01/{occupancy_start,hall_30s,hall_60s,overstay}/
│   ├── EV02/{occupancy_start,hall_30s,hall_60s,overstay}/
│   ├── EV03/{occupancy_start,hall_30s,hall_60s,overstay}/
│   └── EV04/{occupancy_start,hall_30s,hall_60s,overstay}/
data/bestshots/
├── vehicle/
└── plate/
```

DB에는 이미지 BLOB이 아니라 파일 경로와 메타데이터를 저장한다. 네 단계 디렉터리는 미리 생성되며 세션 구분은 파일명의 session_<id>와 DB FK로 유지한다. violation_at 미설정 초기 출차에서는 해당 세션의 DB 경로만 조회해 파일을 삭제하므로, 같은 단계 폴더의 다른 세션 증거는 영향을 받지 않는다.

7. Qt 관제 연동

Pi → Qt MQTT:

```
parking/v1/events/{slot_id}  QoS 1, retain false
parking/v1/state/{slot_id}  QoS 1, retain true
```

주차 이벤트 payload에는 session_id, slot_id, plate_number, vehicle_type, alarm_state, ROI, session_images_url이 포함된다.

Qt → Pi HTTP/HTTPS:

```
GET /api/v1/health
GET /api/v1/parking-slots
GET /api/v1/parking-slots/{slot_id}
GET /api/v1/parking-sessions/active
GET /api/v1/parking-sessions/{session_id}/images
GET /api/v1/images/{image_id}/original
GET /api/v1/images/{image_id}/enhanced
```

인증서와 개인키를 둘 다 설정하면 cpp-httplib SSLServer로 HTTPS를 사용하고, 둘 다 비우면 HTTP로 동작한다. 현재 API token, 로그인, 클라이언트 인증서는 구현되지 않았다.

8. 오류 보고와 동시성

SystemEventReporter는 센서·UART·LoRa 스레드와 SQLite 저장을 분리한다.

```
report()
→ 최대 100개 bounded queue
→ 동일 key 오류 30초 중복 억제
→ worker thread에서 EVENT_LOG 저장
→ DB 실패 시 queue 앞에 넣고 재시도
```

주차 세션 상태, 타이머 만료, VACANT는 전이 mutex로 직렬화한다. 증거 촬영, 30/60초 촬영, OCR, MQTT loop는 각자의 worker thread에서 실행된다. RTSP worker는 Snapshot API 비사용 또는 fallback 활성화 시에만 시작한다.

9. 현재 설정상 주의점

- CAPTURE_SCHED_ENABLED의 기본값은 false라 30/60초 경로는 운영 설정에서 명시적으로 켜야 한다.
- EV01~EV04의 기본 카메라 채널은 모두 ch01이다.
- SQLite와 IVA_EVxx_ROI_* 설정에 ROI가 모두 없으면 해당 슬롯 ROI는 미설정 상태로 유지한다. 촬영/OCR은 전체 프레임을 임의로 사용하지 않고 명확한 오류를 남긴 뒤 건너뛴다.
- 점유 입력은 PARKING_OCCUPANCY_SOURCE=HALL | CAMERA_IVA | HYBRID_OR로 선택한다. 현재 운영값 HYBRID_OR는 입차를 OR로 확정 하되 세션별 센서 확인 상태에 따라 출차 권한을 제한한다. PARKING_HALL_ENABLED는 현재 로드만 되는 구형 설정이다.
- FIRE_ALARM_ENABLED=true이면 SensorLinkManager → FireAlarmManager → Qt MQTT와 ACK 명령 경로가 main.cpp에 배선된다.
- FIRE_UART_*는 구형 설정이며 실제 공유 UART은 SENSOR_UART_*를 사용한다.
- BESTSHOT_ENABLED=false가 기본이며 운영 프로필에서 BestShot은 시작하지 않는다.
- MQTT는 기본 1883 평문 연결이며 username/password/TLS 설정이 없다.
- HTTP API는 TLS를 선택할 수 있지만 API 인증은 없다.
- SystemEventReporter는 UART/LoRa/홀센서에 연결되어 있고 MQTT/RTSP 오류와는 아직 연결되지 않았다.
- /dev/parking_alert 드라이버와 C++ adapter는 존재하지만 타이머 위반 callback에 연결되지 않았다.
- 3~5초 clip 저장은 구현되지 않았다.

10. 핵심 코드 위치

책임	파일
런타임 조립	src/main.cpp
환경 설정	include/app/AppConfig.hpp, src/app/AppConfig.cpp
RTSP 프레임	src/camera/RtspStreamReceiver.cpp, include/camera/CameraChannel.hpp

MQTT 수신/발행	src/mqtt/MqttEventBridge.cpp
홀센서 세션	src/sensor/HallParkingService.cpp
UART/LoRa	src/device/SensorLinkManager.cpp, UartDriver.cpp, LoRaDriver.cpp
30/60초 스케줄	src/parking/CaptureScheduler.cpp, CaptureSchedulerRuntime.cpp
Hall 촬영/OCR 조정	HallCaptureExecutor.cpp, HallCaptureCoordinator.cpp, HallOcrPolicy.cpp
카메라 내부 개선본 수신	CameraSnapshotApiClient.cpp, SnapshotStorage.cpp
시작/장기점유 증거	src/parking/EvidenceCaptureWorker.cpp
OpenCV 저장/전처리	src/snapshot/SnapshotStorage.cpp, src/ocr/PlateImageEnhancer.cpp
Gemini OCR	src/ocr/GeminiOcrClient.cpp, OcrWorker.cpp
DB	src/database/EventDatabase.cpp, EventDatabaseTimer.cpp, db/schema.sql
타이머	src/timer/ParkingSlotManager.cpp, TimerManager.cpp
Qt MQTT event	src/timer/EventManager.cpp, src/main.cpp
Qt HTTP/HTTPS API	src/http/ParkingHttpServer.cpp
IVA	src/event/CameraEventParser.cpp, src/mqtt/MqttEventBridge.cpp
BestShot	src/bestshot/BestShotReceiver.cpp
시스템 오류	src/event/SystemEventReporter.cpp

11. 관련 문서

- docs/ARCHITECTURE_TRACEABILITY.md: 외부 인터페이스와 실제 코드 대응
- docs/CAMERA_MQTT_CAPTURE_PROTOCOL.md: 카메라 촬영 draft 규약과 30/60초 OCR
- docs/HTTP_API.md: Qt 조회 API
- docs/DB_IMPLEMENTATION.md: SQLite 구현
- docs/SENSOR_COMMUNICATION_ERROR_HANDLING.md: 센서·UART·LoRa 오류 처리
- docs/TEST_SCENARIOS.md: 자동·실기기 테스트
- docs/reference/: 승인된 외부 기획서·발표 자료

자동 생성 소스 파일·함수 참조

이 절은 Doxygen XML에서 추출한다. 함수 설명이 비어 있다면 해당 함수에 `/** @brief ... */` 주석을 추가한 후 문서를 다시 생성한다.

driver/parking_alert

driver/parking_alert/parking_alert.c

Linux 커널 드라이버 또는 사용자 공간 ABI를 구현한다.

- MODULE_AUTHOR("VEDA Team 5") — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- MODULE_DESCRIPTION("Smart parking alert character device") — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- MODULE_LICENSE("GPL") — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- MODULE_VERSION("1.0.0") — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- module_exit(parking_alert_exit) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- module_init(parking_alert_init) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- static __poll_t parking_alert_poll(struct file *file, poll_table *wait) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- static int __init parking_alert_init(void) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- static int parking_alert_apply_command(const struct parking_alert_command *command) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `static int parking_alert_open(struct inode *inode, struct file *file)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static int parking_alert_release(struct inode *inode, struct file *file)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static int parking_alert_validate_command(const struct parking_alert_command *command)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static long parking_alert_ioctl(struct file *file, unsigned int command, unsigned long argument)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static ssize_t parking_alert_read(struct file *file, char __user *buffer, size_t count, loff_t *offset)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static ssize_t parking_alert_write(struct file *file, const char __user *buffer, size_t count, loff_t *offset)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static void __exit parking_alert_exit(void)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static void parking_alert_fill_state_locked(struct parking_alert_state *state)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/app

include/app/AppConfig.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `AppConfig app::AppConfig::loadFromEnv()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/app/CallbackLeaseGate.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `Lease & app::CallbackLeaseGate::Lease::operator=(Lease &&other) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `Lease & app::CallbackLeaseGate::Lease::operator=(const Lease &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `app::CallbackLeaseGate::CallbackLeaseGate()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `app::CallbackLeaseGate::Lease::Lease()=default` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `app::CallbackLeaseGate::Lease::Lease(Lease &&other) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `app::CallbackLeaseGate::Lease::Lease(const Lease &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `app::CallbackLeaseGate::Lease::Lease(std::shared_ptr< State > state)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `app::CallbackLeaseGate::Lease::operator bool() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `app::CallbackLeaseGate::Lease::~~Lease()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool app::CallbackLeaseGate::isOpen() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool app::CallbackLeaseGate::waitForDrainedFor(const std::chrono::milliseconds timeout) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< Lease > app::CallbackLeaseGate::tryAcquire() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t app::CallbackLeaseGate::activeCount() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void app::CallbackLeaseGate::Lease::release() noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void app::CallbackLeaseGate::close() noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void app::CallbackLeaseGate::closeAndWait() noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `void app::CallbackLeaseGate::waitForDrained()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/app/RuntimeShutdown.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `RuntimeShutdown & app::RuntimeShutdown::operator=(const RuntimeShutdown &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `app::RuntimeShutdown::RuntimeShutdown(RuntimeShutdownHooks hooks)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `app::RuntimeShutdown::RuntimeShutdown(const RuntimeShutdown &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `app::RuntimeShutdown::~~RuntimeShutdown()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool app::RuntimeShutdown::failed() const` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool app::RuntimeShutdown::shutdown()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool app::RuntimeShutdown::stopped() const` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/bestshot

include/bestshot/BestShotMetadata.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/bestshot/BestShotReceiver.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `BestShotProcessResult bestshot::BestShotReceiver::processOne(PendingMetadata metadata, bool may_requeue)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bestshot::BestShotReceiver::BestShotReceiver(std::vector< std::shared_ptr< camera::CameraChannel > > &channels, parking::ParkingTriggerCoordinator &trigger_coordinator, ocr::OcrWorker &ocr_worker, std::atomic< bool > &running, std::string output_root="data/bestshots", DownloadCallback downloader={}, OcrCallback ocr_callback={}, std::size_t pending_capacity=64)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bestshot::BestShotReceiver::~~BestShotReceiver()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool bestshot::BestShotReceiver::downloadImage(const std::string &rtsp_url, const std::string &image_ref, const std::string &destination)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool bestshot::BestShotReceiver::enqueuePending(PendingMetadata metadata)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string bestshot::BestShotReceiver::evidenceIdentity(const PendingMetadata &metadata)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string bestshot::BestShotReceiver::exactIdentityKey(const PendingMetadata &metadata)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string bestshot::BestShotReceiver::safePathComponent(const std::string &value)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string bestshot::BestShotReceiver::stableDigest(const std::string &value)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< BestShotProcessResult > bestshot::BestShotReceiver::processMetadataDocument(const std::string &camera_id, const std::string &channel_id, const std::string &xml, const std::string &rtsp_url)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `std::vector< BestShotProcessResult > bestshot::BestShotReceiver::reconcilePending()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void bestshot::BestShotReceiver::pendingLoop()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`void bestshot::BestShotReceiver::receiveLoop(const std::shared_ptr< camera::CameraChannel > &channel)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void bestshot::BestShotReceiver::start()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void bestshot::BestShotReceiver::stop()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/camera

include/camera/CameraChannel.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/camera/CameraSnapshotApiClient.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `CameraSnapshotApiClient::HttpResponse camera::CameraSnapshotApiClient::request(const std::string &method, const std::string &url, const std::string &jsonBody, int timeoutMs) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool camera::CameraSnapshotApiClient::configured() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool camera::CameraSnapshotApiClient::discoverChannels()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool camera::CameraSnapshotApiClient::discoverFilters()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool camera::CameraSnapshotApiClient::downloadJpeg(const std::string &path, std::size_t expectedBytes, std::vector< unsigned char > &output)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool camera::CameraSnapshotApiClient::generate(int channel, CameraGeneratedImages &images)` — 같은 카메라 프레임의 original/enhanced JPEG를 생성·다운로드한다.
- `bool camera::CameraSnapshotApiClient::initialize()` — 이미지 서버 시작과 channels/filters discovery를 수행한다.
- `bool camera::CameraSnapshotApiClient::initializeUnlocked()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool camera::CameraSnapshotApiClient::startImageServer()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `camera::CameraSnapshotApiClient::CameraSnapshotApiClient(CameraSnapshotApiConfig config)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const std::string & camera::CameraSnapshotApiClient::lastError() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const std::vector< int > & camera::CameraSnapshotApiClient::channels() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const std::vector< std::string > & camera::CameraSnapshotApiClient::filters() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void camera::CameraSnapshotApiClient::setError(std::string message)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/camera/RtspStreamReceiver.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `bool camera::RtspStreamReceiver::waitForInitialFrames(int timeout_sec)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `camera::RtspStreamReceiver::RtspStreamReceiver(std::vector< std::shared_ptr< CameraChannel > > &channels, int preview_width, int preview_height, int rtsp_retry_delay_ms, int empty_frame_delay_ms, int max_consecutive_read_failures, std::atomic< bool > &running)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void camera::RtspStreamReceiver::captureLoop(std::shared_ptr< CameraChannel > channel)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void camera::RtspStreamReceiver::start()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void camera::RtspStreamReceiver::stop()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/database

include/database/EventDatabase.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `EvidenceInsertResult database::EventDatabase::insertEvidenceImage(std::int64_t session_id, const std::string &original_path, const std::string &evidence_reason, const std::string &captured_at, const std::string &enhanced_path={}, snapshot::NormalizedRoi applied_roi={}, std::uint64_t roi_revision=0)` — 활성 세션에 종류별 증거 이미지 한 장만 원자적으로 연결한다.
- `EvidenceInsertResult database::EventDatabase::insertHallCaptureImage(std::int64_t session_id, const std::string &original_path, const std::string &enhanced_path, const std::string &enhancement_type, const std::string &captured_at, snapshot::NormalizedRoi applied_roi={}, std::uint64_t roi_revision=0)` — 활성 세션에 30/60초 ROI 촬영본을 단계별 최대 한 장 연결한다.
- `VehicleCategory database::EventDatabase::classifyVehicle(std::string_view car_number) const` — VEHICLE의 is_ev/is_phev로 차량 종류를 분류한다.
- `bool database::EventDatabase::acknowledgeFireDelivery(const std::string &delivery_key, std::uint64_t fire_revision)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::EventDatabase::attachEnhancedPlateImage(const std::string &image_path, const std::string &enhanced_image_path)` — 원본 IMAGE_LOG 행에 OpenCV 전처리 파일 경로를 연결한다.
- `bool database::EventDatabase::attachPlateBestShot(int session_id, const std::string &image_path, const std::string &plate_text)` — 번호판 BestShot과 선택적 카메라 plate text를 기존 세션에 연결한다.
- `bool database::EventDatabase::attachVehicleBestShot(int session_id, const std::string &image_path, const std::string &object_id)` — 이미 활성 세션이 있는 슬롯에 차량 BestShot 이미지만 연결한다(세션/슬롯상태는 건드리지 않음).
- `bool database::EventDatabase::cancelUnscheduled(std::int64_t log_id, const std::string &canceled_at)` — DB 생성 뒤 timer enqueue 실패 시 세션을 보상 종료한다.
- `bool database::EventDatabase::completeSlotTransitionEffects(const std::string &command_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::EventDatabase::createEntryWithBestShot(const std::string &slot_id, const std::string &image_path, const std::string &object_id, int *session_id)` — Vehicle BestShot을 근거로 OCCUPIED 슬롯과 ACTIVE 세션을 원자적으로 연결한다.
- `bool database::EventDatabase::createEntryWithSnapshot(const std::string &slot_id, const std::string &image_path, const std::string &source_id, int *session_id)` — 홀센서 입차 Snapshot으로 ACTIVE 세션과 IMAGE/EVENT 로그를 만든다.
- `bool database::EventDatabase::deferSlotTransitionCommand(const std::string &command_id, std::int64_t next_attempt_at_epoch_ms, const std::string &error)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `bool database::EventDatabase::deferSlotTransitionEffects(const std::string &command_id, std::int64_t next_attempt_at_epoch_ms, const std::string &error) noexcept`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::EventDatabase::deleteSessionImageRecords(int session_id)` — 파일 삭제가 끝난 조기 출차 세션의 IMAGE_LOG 행을 모두 제거한다.
- `bool database::EventDatabase::getImage(int image_id, ImageView &row)` — image_id로 이미지 경로와 OCR 메타데이터를 조회한다.
- `bool database::EventDatabase::getParkingSlot(const std::string &slot_id, ParkingSlotView &row)` — slot_id 한 건의 상태를 조회한다.
- `bool database::EventDatabase::insertEvent(const EventRecord &record)` — 정규화된 카메라 이벤트와 선택적 Snapshot을 IMAGE_LOG/EVENT_LOG에 기록한다.
- `bool database::EventDatabase::insertSystemEvent(const std::string &event_type, const std::string &slot_id, const std::string &message)`
— 센서·통신 운영 이벤트를 기존 EVENT_LOG schema에 저장한다.
- `bool database::EventDatabase::isSensorBootIdRetired(const std::string &source_kind, const std::string &sensor_id, const std::string &boot_id) const`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::EventDatabase::listParkingSlots(std::vector< ParkingSlotView > &rows)` — 전체 주차면과 활성 세션을 조회한다.
- `bool database::EventDatabase::listSessionImages(int session_id, std::vector< ImageView > &rows)` — 세션에 연결된 모든 이미지 메타데이터를 조회한다.
- `bool database::EventDatabase::markFireDeliveryInFlight(const std::string &delivery_key, std::uint64_t fire_revision)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::EventDatabase::markFireDeliveryPending(const std::string &delivery_key, std::uint64_t fire_revision, const std::string &error)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::EventDatabase::markPlateOcrUnresolved(std::int64_t session_id, const std::string &slot_id, int attempts)`
— OCR 시도 소진을 UNKNOWN으로 한 번만 EVENT_LOG에 기록한다.
- `bool database::EventDatabase::markViolation(std::int64_t log_id, const std::string &violation_at, const std::string &image_path_2)`
— 아직 ACTIVE인 세션만 VIOLATION으로 조건부 갱신한다.
- `bool database::EventDatabase::occupancySchemaReady() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::EventDatabase::open(const std::string &db_path)` — SQLite 파일을 열고 FK 검사를 활성화한다.
- `bool database::EventDatabase::resetFireInFlightDeliveries()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::EventDatabase::runtimeSchemaReady() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::EventDatabase::upsertSystemSetting(const std::string &key, const std::string &value)`
— 런타임 설정을 원자적으로 추가하거나 갱신한다.
- `database::EventDatabase::EventDatabase()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `database::EventDatabase::EventDatabase(const std::filesystem::path &database_path)` — SQLite 이벤트 DB를 열고 프로토타입에 필요한 연결 옵션을 설정한다.
- `database::EventDatabase::~EventDatabase()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `event::FirestoreMutationResult database::EventDatabase::applyFireStateMutation(const event::FireStateMutation &mutation)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `event::FirestoreMutationResult database::EventDatabase::initializeFireTopology(const std::vector< event::FireChannelBootstrap > &topology)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

```

parking::BestShotAttachResult database::EventDatabase::attachBestShotIfActive(const
parking::CommittedCorrelationLease &lease, parking::BestShotEvidenceKind kind, const std::string
&image_ref, const std::string &image_path, const std::string &plate_text, std::int64_t
• now_epoch_ms)
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
parking::CommittedOccupancyTransition database::EventDatabase::applySlotTransitionCommand(const
• std::string &command_id)
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
parking::ParkingCorrelationMatch database::EventDatabase::resolveParkingCorrelation(const
std::string &camera_id, const std::string &channel_id, const std::string &object_id, std::int64_t
• now_epoch_ms) const
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
parking::SlotAdmissionResult database::EventDatabase::admitSlotTransitionCommand(const
• parking::SlotTransitionCommand &command, std::size_t pending_capacity)
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
std::int64_t database::EventDatabase::createHallSession(const std::string &slot_id, const
• std::string &source_id, const std::string &entry_time)
— 홀센서 입차의 ACTIVE 세션을 만들고 실제 SQLite ID를 반환한다.
std::int64_t database::EventDatabase::insertParked(const std::string &car_number, const std::string
• &slot_id, const std::string &parked_at, const std::string &image_path_1)
— EV/PHEV 장기 점유용 ACTIVE 세션과 최초 이미지를 트랜잭션으로 생성한다.
std::optional< LogRecord > database::EventDatabase::departActiveBySlot(const std::string &slot_id,
• const std::string &departed_at)
— slot_id의 활성 세션을 ENDED로 바꾸고 점유시간을 계산한다.
std::optional< LogRecord > database::EventDatabase::findActiveBySlot(const std::string &slot_id)
• const
— 주차면의 출차되지 않은 최신 세션을 조회한다.
std::optional< LogRecord > database::EventDatabase::findLogById(std::int64_t log_id) const — 불변
session ID로 타이머 읽기 모델을 조회한다.
std::optional< event::FireAlarmStateRecord > database::EventDatabase::getFireAlarmState(const
• std::string &channel_id) const
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
std::optional< event::FireOutboxRecord > database::EventDatabase::getFireDelivery(const std::string
• &delivery_key) const
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
std::optional< std::int64_t > database::EventDatabase::nextScheduledSlotDeadlineEpochMs() const —
Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
std::optional< std::string > database::EventDatabase::findEvidenceImagePath(std::int64_t
• session_id, const std::string &evidence_reason) const
— 이미 저장된 세션 증거 이미지 경로를 조회한다.
std::optional< std::string > database::EventDatabase::getSystemSetting(const std::string &key)
• const
— 런타임 설정 문자열을 조회한다. 키가 없으면 nullptr를 반환한다.
std::size_t database::EventDatabase::admitDueSlotDeadlines(std::int64_t now_epoch_ms, std::size_t
• pending_capacity, const std::vector< std::string > &blocked_slots={})
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
std::size_t database::EventDatabase::pendingSlotTransitionCommandCount() const — Doxygen 설명이 없어 선언
과 호출부를 함께 확인해야 한다.
std::size_t database::EventDatabase::pendingSlotTransitionDrainCount(std::int64_t
• shutdown_cutoff_epoch_ms) const
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
std::size_t database::EventDatabase::pendingSlotTransitionEffectCount() const — Doxygen 설명이 없어 선언과
호출부를 함께 확인해야 한다.

```

- `std::string database::EventDatabase::applyPlateOcr(int session_id, const std::string &slot_id, const std::string &image_path, const std::string &plate_number, double confidence)` — OCR 결과를 저장하고 VEHICLE 조회 결과(EV/NON_EV/UNKNOWN)를 반환한다.
- `std::string database::EventDatabase::readTextFile(const std::filesystem::path &path)` — SQL/config 보조 파일 전체를 문자열로 읽는다.
- `std::vector< LogRecord > database::EventDatabase::listLogs() const` — 타이머 CLI 표시용 전체 세션을 생성 순서로 반환한다.
- `std::vector< event::FireAlarmStateRecord > database::EventDatabase::listFireAlarmStates() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< event::FireOutboxRecord > database::EventDatabase::listFireLifecycleDeliveries() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< event::FireOutboxRecord > database::EventDatabase::listFireRetainedDeliveries() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< event::FireOutboxRecord > database::EventDatabase::listPendingFireLifecycleDeliveries(const std::optional< std::string > &channel_id=std::nullopt) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< parking::CommittedOccupancyTransition > database::EventDatabase::listPendingSlotTransitionEffects(std::int64_t now_epoch_ms) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< parking::DurableSlotCommand > database::EventDatabase::listRunnableSlotTransitionCommands(std::int64_t now_epoch_ms) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< std::pair< std::string, std::string > > database::EventDatabase::listVehicles() const` — 차량번호와 EV/PHEV/NON_EV 문자열 목록을 반환한다.
- `void database::EventDatabase::clearTimerLogs()` — TIMER_ENTRY로 식별되는 데모 타이머 세션만 정리한다.
- `void database::EventDatabase::close()` — 열린 DB 연결을 닫는다.
- `void database::EventDatabase::executeSqlUnlocked(const std::string &sql)` — 준비 과정이 필요 없는 SQL 문자열을 SQLite 연결에서 직접 실행한다.
- `void database::EventDatabase::initialize(const std::filesystem::path &schema_file, const std::filesystem::path &seed_file)` — schema와 seed SQL을 적용하며 구형 컬럼을 먼저 호환 마이그레이션한다.
- `void database::EventDatabase::migrateRuntimeSchema()` — 서버 시작 시 운영 DB에 안전한 먹등 migration만 적용한다.

include/database/SessionTransitionStore.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `bool database::SessionTransitionStore::completeEffects(const std::string &commandId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::SessionTransitionStore::defer(const std::string &commandId, std::int64_t nextAttemptAtEpochMs, const std::string &error) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::SessionTransitionStore::deferEffects(const std::string &commandId, std::int64_t nextAttemptAtEpochMs, const std::string &error) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::SessionTransitionStore::ready() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `database::SessionTransitionStore::SessionTransitionStore(EventDatabase &database) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::CommittedOccupancyTransition database::SessionTransitionStore::apply(const std::string &commandId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- parking::SlotAdmissionResult database::SessionTransitionStore::admit(const parking::SlotTransitionCommand &command, std::size_t pendingCapacity) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::optional< std::int64_t > database::SessionTransitionStore::nextDeadlineEpochMs() const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::size_t database::SessionTransitionStore::admitDueDeadlines(std::int64_t nowEpochMs, std::size_t pendingCapacity, const std::vector< std::string > &blockedSlots={}) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::size_t database::SessionTransitionStore::pendingCount() const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::size_t database::SessionTransitionStore::pendingDrainCount(std::int64_t shutdownCutoffEpochMs) const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::size_t database::SessionTransitionStore::pendingEffectCount() const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::vector< parking::CommittedOccupancyTransition > database::SessionTransitionStore::listPendingEffects(std::int64_t nowEpochMs) const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::vector< parking::DurableSlotCommand > database::SessionTransitionStore::listRunnable(std::int64_t nowEpochMs) const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/database/db_manager.h

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- int db_assign_vehicle_to_session(int session_id, int vehicle_id, const char *plate_number) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- int db_create_parking_session(int vehicle_id, const char *slot_id, const char *plate_number, int *session_id) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- int db_delete_session_images(int session_id) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- int db_end_parking_session(int session_id) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- int db_get_image_by_id(int image_id, DbImageRow *row) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- int db_get_vehicle_by_plate(const char *plate_number, int *vehicle_id, int *is_ev, int *is_phev) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- int db_insert_event_log(int session_id, const char *slot_id, const char *event_type, const char *message) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- int db_insert_image_log(int session_id, const char *original_path, const char *enhanced_path, const char *enhancement_type, const char *ocr_result) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- int db_insert_image_log_with_roi(int session_id, const char *original_path, const char *enhanced_path, const char *enhancement_type, const char *ocr_result, double roi_x, double roi_y, double roi_width, double roi_height, unsigned long long roi_revision) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- int db_mark_event_handled(int event_id) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- int db_open(const char *path) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- int db_update_image_enhanced_by_path(const char *original_path, const char *enhanced_path) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- int db_update_image_ocr_by_path(const char *original_path, const char *ocr_result) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- int db_update_slot_status(const char *slot_id, const char *status) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `int db_visit_parking_slots(const char *slot_id, DbParkingSlotVisitor visitor, void *context)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `int db_visit_session_images(int session_id, DbImageVisitor visitor, void *context)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `struct sqlite3 * db_native_handle(void)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void db_close(void)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/device

include/device/LinuxDriverAdapter.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `LinuxDriverAdapter & device::LinuxDriverAdapter::operator=(LinuxDriverAdapter &&other)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `LinuxDriverAdapter & device::LinuxDriverAdapter::operator=(const LinuxDriverAdapter &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `ParkingAlertState device::LinuxDriverAdapter::state() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool device::LinuxDriverAdapter::isOpen() const` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const std::string & device::LinuxDriverAdapter::devicePath() const` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `device::LinuxDriverAdapter::LinuxDriverAdapter(LinuxDriverAdapter &&other)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `device::LinuxDriverAdapter::LinuxDriverAdapter(const LinuxDriverAdapter &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `device::LinuxDriverAdapter::LinuxDriverAdapter(std::string devicePath="/dev/parking_alert")` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `device::LinuxDriverAdapter::~~LinuxDriverAdapter()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::LinuxDriverAdapter::clearAll()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::LinuxDriverAdapter::clearSlot(std::uint32_t slotIndex, std::uint64_t eventId=0)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::LinuxDriverAdapter::closeDevice()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::LinuxDriverAdapter::openDevice()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::LinuxDriverAdapter::requireOpen() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::LinuxDriverAdapter::setSlot(std::uint32_t slotIndex, std::uint64_t eventId=0)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::LinuxDriverAdapter::updateSlot(unsigned long request, std::uint16_t operation, std::uint32_t slotIndex, std::uint64_t eventId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/device/LoRaDriver.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `bool device::LoRaDriver::send(const LoRaFrame &frame, std::string *error=nullptr)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool device::LoRaDriver::sendTo(const LoRaDestination &destination, const LoRaFrame &frame, std::string *error=nullptr)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `device::LoRaDriver::LoRaDriver(UartDriver &uart)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::uint16_t device::LoRaDriver::crc16Ccitt(std::span< const std::uint8_t > bytes)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::uint64_t device::LoRaDriver::rejectedFrames() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `std::vector< LoRaFrame > device::LoRaDriver::consume(std::span< const std::uint8_t > bytes)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< std::uint8_t > device::LoRaDriver::encode(const LoRaFrame &frame)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/device/SensorLinkManager.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `SensorLinkManager & device::SensorLinkManager::operator=(const SensorLinkManager &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `SensorLinkMode device::SensorLinkManager::parseMode(const std::string &value)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool device::SensorLinkManager::connected() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool device::SensorLinkManager::running() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool device::SensorLinkManager::sendAlertCommand(const std::string &command, std::uint32_t sequence, std::string *error=nullptr)` — STM32/LoRa 반대 방향으로 경고 명령을 전송한다.
- `bool device::SensorLinkManager::start()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool device::SensorLinkManager::waitReconnect()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `device::SensorLinkManager::SensorLinkManager(Config config, SensorLineHandler handler, event::SystemEventReporter *reporter=nullptr)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `device::SensorLinkManager::~SensorLinkManager(const SensorLinkManager &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `device::SensorLinkManager::~~SensorLinkManager()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string device::SensorLinkManager::modeName(SensorLinkMode mode)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::SensorLinkManager::consumeLoRaFrames(const std::uint8_t *data, std::size_t size)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::SensorLinkManager::consumeUartLines(const std::uint8_t *data, std::size_t size)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::SensorLinkManager::report(event::SystemEventCode code, event::SystemEventSeverity severity, const std::string &message, std::uint32_t retry_count=0, bool recovered=false) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::SensorLinkManager::run()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::SensorLinkManager::stop()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/device/UartDriver.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `UartDriver & device::UartDriver::operator=(const UartDriver &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool device::UartDriver::connect(std::string *error=nullptr)` — UART 장치를 non-blocking raw 8N1 모드로 연다.
- `bool device::UartDriver::connected() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool device::UartDriver::writeAll(std::span< const std::uint8_t > data, std::string *error=nullptr)` — 전체 byte가 전송될 때까지 partial write를 처리한다.
- `const Config & device::UartDriver::config() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `device::UartDriver::UartDriver(Config config)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `device::UartDriver::UartDriver(const UartDriver &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `device::UartDriver::~~UartDriver()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `int device::UartDriver::readSome(std::span< std::uint8_t > output, std::string *error=nullptr)` — poll 후 수신 가능한 byte를 읽는다.
- `void device::UartDriver::disconnect()` — 열려 있는 descriptor를 닫는다.

include/event

include/event/CameraEvent.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/event/CameraEventParser.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `CameraEvent event::CameraEventParser::parse(const std::string &raw_topic, const std::string &raw_payload, const std::string &default_channel_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::CameraEventParser::parseEventChannelId(const std::string &topic, const std::string &default_channel_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::CameraEventParser::parseEventType(const std::string &topic, const std::string &payload)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::CameraEventParser::parseSeverity(const std::string &event_type)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::CameraEventParser::parseSourceId(const std::string &topic)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::CameraEventParser::parseVideoSourceToken(const std::string &topic)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< CameraEvent > event::CameraEventParser::parseMany(const std::string &raw_topic, const std::string &raw_payload, const std::string &default_channel_id)` — 한 MQTT payload의 NotificationMessage들을 개별 이벤트로 분리한다.

include/event/EventPayloadBuilder.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `std::string event::EventPayloadBuilder::buildFireEventId(const FireSignal &signal)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::EventPayloadBuilder::buildFireJson(const std::string &camera_id, const std::string &channel_id, const FireSignal &signal, FireAlarmLifecycle lifecycle, const std::string &event_id, const std::string &alarm_id, std::uint64_t fire_revision, const std::string &delivery_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::EventPayloadBuilder::buildJson(const std::string &camera_id, const std::string &channel_id, const CameraEvent &event, const std::string &snapshot_path, const parking::AppliedParkingRoi *applied_roi=nullptr)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/event/FireAlarmEvent.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/event/FireAlarmService.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `FireAlarmService & event::FireAlarmService::operator=(const FireAlarmService &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `FireCommandResult event::FireAlarmService::enqueue(Command command)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `FireCommandResult event::FireAlarmService::process(const Command &command)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `FireCommandResult event::FireAlarmService::processAcknowledge(const Command &command)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `FireCommandResult event::FireAlarmService::processSignal(const Command &command, const FireChannelBinding &binding)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `FireCommandResult event::FireAlarmService::submitAcknowledge(std::string channel_id, std::string alarm_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `FireCommandResult event::FireAlarmService::submitSignal(FireSignal signal)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `FireStoreMutationResult event::FireAlarmService::applyTransition(const FireStateMutation &mutation)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireAlarmService::configurationValid() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireAlarmService::drainCommitted(std::chrono::milliseconds timeout)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireAlarmService::initialize()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireAlarmService::start()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireAlarmService::stop(std::chrono::milliseconds timeout=std::chrono::seconds(30))` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireBindingParseResult::valid() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const FireChannelBinding * event::FireAlarmService::findByChannel(const std::string &channel_id)`
- `const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const FireChannelBinding * event::FireAlarmService::findBySensor(const std::string &sensor_id)`
- `const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `event::FireAlarmService::FireAlarmService(const FireAlarmService &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `event::FireAlarmService::FireAlarmService(database::EventDatabase &database, Config config, std::vector< FireChannelBinding > bindings, CommitObserver observer={}, AckReadiness ack_readiness={}, MutationBegin mutation_begin={}, MutationEnd mutation_end={})` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `event::FireAlarmService::~FireAlarmService()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t event::FireAlarmService::bindingCount() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::FireAlarmService::configurationError() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireAlarmService::closeIngress() noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireAlarmService::notify(const FireCommandResult &result) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireAlarmService::run() noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/event/FireDeliveryCoordinator.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

```
FireDeliveryCoordinator & event::FireDeliveryCoordinator::operator=(const FireDeliveryCoordinator
• &)=delete
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• auto event::FireDeliveryCoordinator::AttemptKey::operator<=>(const AttemptKey &) const =default —
Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• bool event::FireDeliveryCoordinator::allChannelsReady() const noexcept — Doxygen 설명이 없어 선언과 호출부를
함께 확인해야 한다.
• bool event::FireDeliveryCoordinator::beginDomainMutation(const std::string &channel_id) noexcept —
Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• bool event::FireDeliveryCoordinator::beginLifecycleRelease(std::uint64_t observed_generation) —
Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• bool event::FireDeliveryCoordinator::drainDeliveries(std::chrono::milliseconds timeout) — Doxygen 설명
이 없어 선언과 호출부를 함께 확인해야 한다.
bool event::FireDeliveryCoordinator::enqueueTransportFact(const mqtt::MqttTransportFact &fact)
• noexcept
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• bool event::FireDeliveryCoordinator::hasLifecycleInFlight(const std::string &channel_id) const —
Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• bool event::FireDeliveryCoordinator::hasRetainedInFlight(const std::string &channel_id) const —
Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• bool event::FireDeliveryCoordinator::isReady(const std::string &channel_id) const noexcept — Doxygen
설명이 없어 선언과 호출부를 함께 확인해야 한다.
bool event::FireDeliveryCoordinator::isRegularEgressGrantCurrent(const RegularEgressGrant &grant)
• const noexcept
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
bool event::FireDeliveryCoordinator::isRevisionSynchronized(const std::string &channel_id,
• std::uint64_t fire_revision) const noexcept
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• bool event::FireDeliveryCoordinator::publish(const FireOutboxRecord &delivery) — Doxygen 설명이 없어 선언
과 호출부를 함께 확인해야 한다.
bool event::FireDeliveryCoordinator::retryDeferred(const std::string &channel_id,
• std::chrono::steady_clock::time_point now) const
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• bool event::FireDeliveryCoordinator::start() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• bool event::FireDeliveryCoordinator::stop() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• bool event::FireDeliveryCoordinator::waitUntilReady(std::chrono::milliseconds timeout) — Doxygen 설명
이 없어 선언과 호출부를 함께 확인해야 한다.
• event::FireDeliveryCoordinator::FireDeliveryCoordinator(const FireDeliveryCoordinator &)=delete —
Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
event::FireDeliveryCoordinator::FireDeliveryCoordinator(database::EventDatabase &database, Config
• config, TrackedPublisher publisher, EpochFaultHandler fault_handler={})
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• event::FireDeliveryCoordinator::~~FireDeliveryCoordinator() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한
다.
• std::chrono::steady_clock::time_point event::FireDeliveryCoordinator::nextWakeAt() const — Doxygen 설
명이 없어 선언과 호출부를 함께 확인해야 한다.
std::optional< FireDeliveryCoordinator::RegularEgressGrant >
• event::FireDeliveryCoordinator::beginRegularEgress() noexcept
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
```

- `void event::FireDeliveryCoordinator::clearRetry(const std::string &channel_id) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::completeDomainMutation(const std::string &channel_id) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::completeRegularEgress()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`void event::FireDeliveryCoordinator::disconnectEpoch(mqtt::MqttConnectionEpoch epoch, const std::string &reason)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::endLifecycleRelease()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::expirePubAckDeadline()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::failCloseAllReadinessLocked()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::invalidateRegularEgressLocked()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::notifyOutboxChanged(const std::string &channel_id) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::processFact(const mqtt::MqttTransportFact &fact)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::pump()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`void event::FireDeliveryCoordinator::recomputeReadiness(const std::vector< FireOutboxRecord > &retained_rows, std::uint64_t observed_generation)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::recoverActorFailure(const std::string &reason) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::requeueAllInflight(const std::string &reason)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::run()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::scheduleRetry(const std::string &channel_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::updateIdleState()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/event/FirePersistence.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `bool event::FireStoreMutationResult::committed() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/event/IvaEventResolver.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `std::optional< IvaResolvedTarget > event::IvaEventResolver::resolve(const std::string &cameraId, const CameraEvent &cameraEvent, const std::vector< parking::ParkingSlotConfig > &slots, const std::vector< app::IvaAreaConfig > &areas, std::string *error=nullptr)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/event/IvaOccupancyCoordinator.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

```

IvaCoordinationResult event::IvaOccupancyCoordinator::handle(IvaOccupancySignal signal,
• std::chrono::steady_clock::time_point now=std::chrono::steady_clock::now())
— IVA 액션 한 건을 점유 전이 또는 출차 예약으로 반영한다.
event::IvaOccupancyCoordinator::IvaOccupancyCoordinator(const std::vector<
• parking::ParkingSlotConfig > &slots, std::chrono::milliseconds exitConfirmDelay)
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
std::optional< std::chrono::steady_clock::time_point >
• event::IvaOccupancyCoordinator::nextDeadline() const
— 가장 빠른 출차 확인 시각을 반환한다.
std::vector< IvaOccupancySignal >
• event::IvaOccupancyCoordinator::takeDue(std::chrono::steady_clock::time_point now)
— 확인 시각이 지난 출차 신호를 큐에서 제거해 반환한다.

```

include/event/IvaSlotOccupancyAggregator.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

```

event::IvaSlotOccupancyAggregator::IvaSlotOccupancyAggregator(const std::vector<
• parking::ParkingSlotConfig > &slots)
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
std::optional< IvaSlotOccupancyUpdate > event::IvaSlotOccupancyAggregator::update(const std::string
&slotId, const std::string &cameraId, const std::string &videoSourceToken, const std::string
• &ruleName, bool active)
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
std::string event::IvaSlotOccupancyAggregator::observationKey(const std::string &cameraId, const
• std::string &videoSourceToken, const std::string &ruleName)
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

```

include/event/SystemEventReporter.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

```

• SystemEventReporter & event::SystemEventReporter::operator=(const SystemEventReporter &)=delete —
Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• bool event::SystemEventReporter::persist(const SystemEvent &event) noexcept — Doxygen 설명이 없어 선언과 호
출부를 함께 확인해야 한다.
• bool event::SystemEventReporter::running() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한
다.
• bool event::SystemEventReporter::start() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• bool event::SystemEventReporter::stop() noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• event::SystemEventReporter::SystemEventReporter(Sink sink) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한
다.
• event::SystemEventReporter::SystemEventReporter(Sink sink, Config config) — Doxygen 설명이 없어 선언과 호출
부를 함께 확인해야 한다.
• event::SystemEventReporter::SystemEventReporter(const SystemEventReporter &)=delete — Doxygen 설명이 없
어 선언과 호출부를 함께 확인해야 한다.
• event::SystemEventReporter::~SystemEventReporter() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• std::size_t event::SystemEventReporter::queuedCount() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함
게 확인해야 한다.
• std::string event::SystemEventReporter::deduplicationKey(const SystemEvent &event) — Doxygen 설명이 없어
선언과 호출부를 함께 확인해야 한다.
• void event::SystemEventReporter::clearRecoveredStateLocked(const SystemEvent &event) — Doxygen 설명이
없어 선언과 호출부를 함께 확인해야 한다.
• void event::SystemEventReporter::enqueueLocked(SystemEvent event) — Doxygen 설명이 없어 선언과 호출부를 함께 확
인해야 한다.

```

- `void event::SystemEventReporter::report(SystemEvent event) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::SystemEventReporter::run() noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/http

include/http/ParkingHttpServer.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `HttpServerState http::ParkingHttpServer::state() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `ParkingHttpServer & http::ParkingHttpServer::operator=(const ParkingHttpServer &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool http::ParkingHttpServer::start()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`bool http::ParkingHttpServer::stop(std::chrono::milliseconds timeout=std::chrono::seconds(30))`
- `noexcept`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool http::ParkingHttpServer::usesTls() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `http::ParkingHttpServer::ParkingHttpServer(const ParkingHttpServer &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`http::ParkingHttpServer::ParkingHttpServer(database::EventDatabase &database, ServerConfig config, settings::OverstayThresholdService *overstay_settings=nullptr, settings::ParkingRoiSettingsService *roi_settings=nullptr)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `http::ParkingHttpServer::~ParkingHttpServer()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void http::ParkingHttpServer::closeIngress()` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void http::ParkingHttpServer::closeIngressLocked()` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void http::ParkingHttpServer::registerRoutes()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void http::ParkingHttpServer::stopListenerLocked()` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/mqtt

include/mqtt/MqttEndpoint.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `MqttEndpoint & mqtt::MqttEndpoint::operator=(const MqttEndpoint &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `MqttEndpointState mqtt::MqttEndpoint::state() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`MqttPublishResult mqtt::MqttEndpoint::publish(const std::string &topic, const std::string &payload,`
- `int qos, bool retain)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`MqttPublishResult mqtt::MqttEndpoint::publishInEpoch(const std::string &topic, const std::string`
- `&payload, int qos, bool retain, MqttConnectionEpoch expectedEpoch)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`MqttTrackedPublishResult mqtt::MqttEndpoint::publishTracked(const std::string &topic, const`
- `std::string &payload, bool retain, MqttPublishCorrelation correlation, MqttConnectionEpoch`
`expectedEpoch)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::abortActiveEpoch(MqttConnectionEpoch expectedEpoch) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `bool mqtt::MqttEndpoint::bindApplicationHandler(ApplicationHandler handler)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::bindTransportObservers(TransportObservers observers)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::quiesceIngress(std::chrono::milliseconds timeout=std::chrono::seconds(30))`
- `noexcept`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::quiesceIngressLocked(std::chrono::milliseconds timeout) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::quiesceTransportObservers(std::chrono::milliseconds`
- `timeout=std::chrono::seconds(30)) noexcept`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::quiesceTransportObserversLocked(std::chrono::milliseconds timeout)`
- `noexcept`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::start(const MqttConnectionOptions &options, const std::vector<`
- `MqttSubscription > &subscriptions)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::stop(std::chrono::milliseconds timeout=std::chrono::seconds(30)) noexcept`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::stopLocked(std::chrono::milliseconds timeout) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `mqtt::MqttEndpoint::MqttEndpoint(const MqttEndpoint &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `mqtt::MqttEndpoint::MqttEndpoint(std::unique_ptr< IMqttTransport > transport)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `mqtt::MqttEndpoint::~MqttEndpoint()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void mqtt::MqttEndpoint::closeIngress()` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void mqtt::MqttEndpoint::closeIngressLocked()` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void mqtt::MqttEndpoint::dispatchApplication(const std::string &topic, const std::string &payload)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void mqtt::MqttEndpoint::dispatchConnected()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void mqtt::MqttEndpoint::dispatchDisconnected(int reason)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void mqtt::MqttEndpoint::dispatchPublished(int messageId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void mqtt::MqttEndpoint::dispatchTransportFact(const MqttTransportFact &fact)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/mqtt/MqttEventBridge.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `MqttConnectionEpoch mqtt::RegularEgressPermit::epoch()` `const` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `MqttEndpointState mqtt::MqttEventBridge::state()` `const` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `MqttEventBridge & mqtt::MqttEventBridge::operator=(const MqttEventBridge &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `MqttTrackedPublishResult mqtt::MqttEventBridge::publishTrackedFire(const std::string &topic, const std::string &payload, bool retain, MqttPublishCorrelation correlation, MqttConnectionEpoch`
- `expectedEpoch)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `RegularEgressPermit & mqtt::RegularEgressPermit::operator=(RegularEgressPermit &&other) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `RegularEgressPermit & mqtt::RegularEgressPermit::operator=(const RegularEgressPermit &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::abortActiveEpoch(MqttConnectionEpoch expectedEpoch) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::bindCameraSnapshotGenerator(CameraSnapshotGenerator generator)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::bindFireAckHandler(FireAckHandler handler)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::bindIvaOccupancyHandler(IvaOccupancyHandler handler)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::bindParkingRoiResolver(RoiResolver resolver)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::bindRegularEgressAdmission(RegularEgressAdmission admission)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::bindSensorMessageHandler(SensorMessageHandler handler)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::bindTransportFactHandler(TransportFactHandler handler)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::publish(const std::string &topic, const std::string &payload, int qos=1, bool retain=false)` — 화재 알림 등 상위 계층의 일반 MQTT 메시지를 발행한다.
- `bool mqtt::MqttEventBridge::publishApplicationEvent(const std::string &topic, const std::string &payload, int qos=1, bool retain=false)` — 카메라 촬영 요청 등 서버 application 메시지를 발행한다.
- `bool mqtt::MqttEventBridge::publishQtEvent(const std::string &topic, const std::string &payload, int qos=1, bool retain=false)` — Qt 관제 클라이언트용 상태 이벤트를 발행한다.
- `bool mqtt::MqttEventBridge::quiesceIngress(std::chrono::milliseconds timeout=std::chrono::seconds(30))` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::quiesceTransportObservers(std::chrono::milliseconds timeout=std::chrono::seconds(30))` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::start()` — Broker 연결, topic 구독과 network loop를 시작한다.
- `bool mqtt::MqttEventBridge::stop(std::chrono::milliseconds timeout=std::chrono::seconds(30))` — Mosquitto loop와 연결을 종료하고 자원을 해제한다.
- `bool mqtt::RegularEgressPermit::isCurrent() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `mqtt::MqttEventBridge::~MqttEventBridge(const MqttEventBridge &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `mqtt::MqttEventBridge::MqttEventBridge(const app::AppConfig &config, std::vector< std::shared_ptr< camera::CameraChannel > > &channels, database::EventDatabase &database, snapshot::SnapshotStorage &snapshot_storage, ocr::OcrWorker &ocr_worker, std::vector< parking::ParkingSlotConfig > parking_slot_configs, SensorMessageHandler sensor_message_handler={}, FireAckHandler fire_ack_handler={}, IvaOccupancyHandler iva_occupancy_handler={}, std::unique_ptr< IMqttTransport > transport=makeMosquittoTransport(), notification::TelegramChannelNotifier *telegram_notifier=nullptr)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `mqtt::MqttEventBridge::~~MqttEventBridge()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `mqtt::RegularEgressPermit::RegularEgressPermit()=default` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `mqtt::RegularEgressPermit::RegularEgressPermit(RegularEgressPermit &&other) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `mqtt::RegularEgressPermit::RegularEgressPermit(const MqttConnectionEpoch epoch, const std::uint64_t generation, std::function< bool()> validate, std::function< void()> release)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `mqtt::RegularEgressPermit::RegularEgressPermit(const RegularEgressPermit &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `mqtt::RegularEgressPermit::operator bool() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `mqtt::RegularEgressPermit::~RegularEgressPermit()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::uint64_t mqtt::RegularEgressPermit::generation() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void mqtt::MqttEventBridge::closeIngress() noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void mqtt::MqttEventBridge::onMessage(const std::string &topic, const std::string &payload)` — 수신 메시지를 정규화하고 이벤트별 처리 흐름을 실행한다.
- `void mqtt::MqttEventBridge::processCameraEvent(event::CameraEvent camera_event)` — 분리된 카메라 이벤트를 기존 IVA/일반 이벤트 흐름으로 처리한다.
- `void mqtt::RegularEgressPermit::reset() noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/mqtt/MqttTransport.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `mqtt::IMqttTransport::ObserverCallbacks::ObserverCallbacks()=default` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `mqtt::IMqttTransport::ObserverCallbacks::ObserverCallbacks(std::function< void()> connected, std::function< void(int)> disconnected, std::function< void(int)> published, std::function< void(const MqttTransportFact &)> fact={})` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `mqtt::MqttTrackedPublishToken::operator bool() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `virtual MqttPublishResult mqtt::IMqttTransport::publish(const std::string &topic, const std::string &payload, int qos, bool retain)=0` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `virtual MqttPublishResult mqtt::IMqttTransport::publishInEpoch(const std::string &topic, const std::string &payload, int qos, bool retain, MqttConnectionEpoch expectedEpoch)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `virtual MqttTrackedPublishResult mqtt::IMqttTransport::publishTracked(const std::string &topic, const std::string &payload, bool retain, MqttPublishCorrelation correlation, MqttConnectionEpoch expectedEpoch)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `virtual bool mqtt::IMqttTransport::abortActiveEpoch(MqttConnectionEpoch expectedEpoch) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `virtual bool mqtt::IMqttTransport::start(const MqttConnectionOptions &options, const std::vector< MqttSubscription > &subscriptions, MessageCallback messageCallback, ObserverCallbacks observerCallbacks)=0` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `virtual bool mqtt::IMqttTransport::stopAndJoin() noexcept=0` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `virtual mqtt::IMqttTransport::~IMqttTransport()=default` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/notification

include/notification/TelegramApiClient.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `bool notification::TelegramApiClient::isEnabled()` `const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool notification::TelegramApiClient::sendMessage(const std::string &text)` `const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const std::string & notification::TelegramApiClient::channel()` `const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `notification::TelegramApiClient::TelegramApiClient(const Config &config)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/notification/TelegramChannelNotifier.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `TelegramChannelNotifier & notification::TelegramChannelNotifier::operator=(const TelegramChannelNotifier &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool notification::TelegramChannelNotifier::enqueue(TelegramMessage message)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool notification::TelegramChannelNotifier::running()` `const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool notification::TelegramChannelNotifier::sendWithRetry(const TelegramMessage &message, const std::string &text)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool notification::TelegramChannelNotifier::start()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `notification::TelegramChannelNotifier::TelegramChannelNotifier(const Config &config, const TelegramApiClient &api_client)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `notification::TelegramChannelNotifier::TelegramChannelNotifier(const TelegramChannelNotifier &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `notification::TelegramChannelNotifier::~~TelegramChannelNotifier()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void notification::TelegramChannelNotifier::run()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void notification::TelegramChannelNotifier::stop()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/notification/TelegramMessage.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/notification/TelegramMessageFormatter.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `std::string notification::TelegramMessageFormatter::format(const TelegramMessage &message)` `const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string notification::TelegramMessageFormatter::truncate(const std::string &text)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/ocr

include/ocr/GeminiOcrClient.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `OcrResult ocr::GeminiOcrClient::recognizePlate(const std::string &image_path, const std::string &enhanced_image_path="") const`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `OcrResult ocr::GeminiOcrClient::recognizePlateWithModel(const std::string &model, const std::string &image_path, const std::string &enhanced_image_path) const`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool ocr::GeminiOcrClient::configured() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool ocr::OcrResult::retryable() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `ocr::GeminiOcrClient::GeminiOcrClient(std::string api_key, std::string model, long connect_timeout_sec, long request_timeout_sec, std::string fallback_model="", HttpTransport transport={})`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/ocr/OcrWorker.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `bool ocr::OcrWorker::enabled() const` — Gemini client가 실제 요청 가능한 상태인지 반환한다.
- `ocr::OcrWorker::OcrWorker(GeminiOcrClient client, database::EventDatabase &database, bool preprocess_enabled, ResultCallback result_callback={})`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `ocr::OcrWorker::~~OcrWorker()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void ocr::OcrWorker::cancelSession(int session_id)` — 조기 출차 세션의 대기/진행 OCR 결과가 DB와 타이머에 반영되지 않게 한다.
- `void ocr::OcrWorker::enqueue(int session_id, const std::string &slot_id, const std::string &image_path, const std::string &enhanced_image_path={})`
— BestShot 이미지를 기존 session의 OCR 작업으로 등록한다.
- `void ocr::OcrWorker::enqueueHallCapture(const HallCaptureTask &task)` — 30/60초 홀 촬영본을 전용 결과 callback이 있는 OCR 작업으로 등록한다.
- `void ocr::OcrWorker::enqueueScene(const std::string &slot_id, const std::string &image_path, const std::string &enhanced_image_path)`
— 세션이 아직 없는 IVA scene을 후보 탐색 OCR 작업으로 등록한다.
- `void ocr::OcrWorker::process(const Task &task, HallCaptureResult &hall_result)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void ocr::OcrWorker::run()` — queue 대기, 전처리, OCR, 정규화와 DB 반영을 반복한다.
- `void ocr::OcrWorker::setHallCaptureCallback(HallCaptureCallback callback)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void ocr::OcrWorker::start()` — OCR worker thread를 시작한다.
- `void ocr::OcrWorker::stop()` — 남은 worker를 깨워 종료하고 join한다.

include/ocr/PlateImageEnhancer.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/ocr/PlateNormalizer.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/parking

include/parking/ActiveParkingSessionIndex.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `std::optional< std::string > parking::ActiveParkingSessionIndex::findActiveSessionId(const std::string &slotId) const`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t parking::ActiveParkingSessionIndex::size() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::ActiveParkingSessionIndex::apply(const ParkingTransitionResult &transition)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::ActiveParkingSessionIndex::clear()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`void parking::ActiveParkingSessionIndex::clearActive(const std::string &slotId, const std::string &expectedParkingSessionId={})`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::ActiveParkingSessionIndex::setActive(const std::string &slotId, const std::string &parkingSessionId)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/AppliedParkingRoi.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/parking/CaptureRequest.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `const char * parking::toReasonString(const CaptureReason reason) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/CaptureScheduler.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `CaptureRequest parking::CaptureScheduler::buildRequest(const std::string &sessionId, const SessionState &session, const CaptureState &capture) const`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `CaptureScheduler::ScheduleReport parking::CaptureScheduler::onTransition(const ParkingTransitionResult &transition)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `DispatchOutcome parking::CaptureScheduler::onDispatchResult(const CaptureRequest &request, bool accepted, std::chrono::steady_clock::time_point now)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::CaptureScheduler::CaptureScheduler(CaptureSchedulerConfig config, CaptureTargetResolver &resolver)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< std::chrono::steady_clock::time_point > parking::CaptureScheduler::nextDeadline()`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t parking::CaptureScheduler::trackedSessions() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

```
std::vector< CaptureRequest > parking::CaptureScheduler::due(std::chrono::steady_clock::time_point  
• now)
```

— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/CaptureSchedulerRuntime.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

```
CaptureSchedulerRuntime & parking::CaptureSchedulerRuntime::operator=(const CaptureSchedulerRuntime  
• &)=delete  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
parking::CaptureSchedulerRuntime::CaptureSchedulerRuntime(CaptureScheduler &scheduler,  
• CapturePublisher publisher)  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
• parking::CaptureSchedulerRuntime::CaptureSchedulerRuntime(const CaptureSchedulerRuntime &)=delete —  
Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
• parking::CaptureSchedulerRuntime::~CaptureSchedulerRuntime() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야  
한다.  
• void parking::CaptureSchedulerRuntime::onTransition(const ParkingTransitionResult &transition) —  
Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
• void parking::CaptureSchedulerRuntime::run() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
• void parking::CaptureSchedulerRuntime::start() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
• void parking::CaptureSchedulerRuntime::stop() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
```

include/parking/EvidenceCaptureWorker.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

```
EvidenceCaptureWorker & parking::EvidenceCaptureWorker::operator=(const EvidenceCaptureWorker  
• &)=delete  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
bool parking::EvidenceCaptureWorker::Later::operator()(const Job &left, const Job &right) const  
• noexcept  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
• bool parking::EvidenceCaptureWorker::canceled(std::int64_t session_id) const — Doxygen 설명이 없어 선언과  
호출부를 함께 확인해야 한다.  
• bool parking::EvidenceCaptureWorker::expediteOverstay(std::int64_t session_id) — 타이머가 먼저 만료되면 기  
존 초과 증거 작업을 즉시 실행 대상으로 만든다.  
• bool parking::EvidenceCaptureWorker::restoreSession(EvidenceCaptureRequest request) — 재시작 시 DB에 없  
는 증거만 원래 T0 기준으로 다시 예약한다.  
• bool parking::EvidenceCaptureWorker::scheduleSession(EvidenceCaptureRequest request) — 시작 즉시 한 장과  
T0+지연 한 장을 세션당 한 번 예약한다.  
bool parking::EvidenceCaptureWorker::scheduleSessionImpl(EvidenceCaptureRequest request, bool  
• include_start, bool include_overstay, bool restored)  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
• bool parking::EvidenceCaptureWorker::start() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
• parking::EvidenceCaptureWorker::EvidenceCaptureWorker(const EvidenceCaptureWorker &)=delete —  
Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
parking::EvidenceCaptureWorker::EvidenceCaptureWorker(snapshot::SnapshotStorage &storage,  
database::EventDatabase &database, Config config, Completion completion={}, Capture capture={},  
• RoiResolver roi_resolver={})  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
• parking::EvidenceCaptureWorker::~EvidenceCaptureWorker() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
• std::size_t parking::EvidenceCaptureWorker::pendingCount() const — Doxygen 설명이 없어 선언과 호출부를 함께 확  
인해야 한다.
```

- `std::size_t parking::EvidenceCaptureWorker::updateOverstayDelay(std::chrono::milliseconds delay)` — 모든 활성 세션의 초과 증거 deadline을 같은 TO 기준으로 재계산한다.
- `void parking::EvidenceCaptureWorker::cancelSession(std::int64_t session_id)` — VACANT 세션의 아직 실행되지 않은 작업을 취소한다.
- `void parking::EvidenceCaptureWorker::emit(EvidenceCaptureResult result) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::EvidenceCaptureWorker::process(Job job) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::EvidenceCaptureWorker::run()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::EvidenceCaptureWorker::stop()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/HallCaptureCoordinator.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `CaptureImageResult parking::HallCaptureCoordinator::onCaptureImage(const CapturedImage &image)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::HallCaptureCoordinator::HallCaptureCoordinator(HallCapturePorts ports, int maxOcrAttempts=2)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< std::int64_t > parking::HallCaptureCoordinator::parseSessionId(const std::string &value) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t parking::HallCaptureCoordinator::trackedSessions()` const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::HallCaptureCoordinator::onOcrOutcome(const HallOcrOutcome &outcome)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::HallCaptureCoordinator::onTransition(const ParkingTransitionResult &transition)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/HallCaptureExecutor.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `PlateIlluminator::Scope parking::HallCaptureExecutor::illuminate(const CaptureRequest &request)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool parking::HallCaptureExecutor::execute(const CaptureRequest &request) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::HallCaptureExecutor::HallCaptureExecutor(std::vector< std::shared_ptr< camera::CameraChannel > > &channels, snapshot::SnapshotStorage &storage, HallCaptureCoordinator &coordinator, DraftPublisher draftPublisher, camera::CameraSnapshotApiClient *snapshotApiClient=nullptr, bool rtspFallback=false, PlateIlluminator *illuminator=nullptr, RoiResolver roiResolver={})` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/HallCaptureTypes.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `CaptureStage parking::toStage(const CaptureReason reason) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const char * parking::toEnhancementType(const CaptureStage stage) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/HallOcrPolicy.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `HallOcrPolicy::CloseReport parking::HallOcrPolicy::closeSession(const std::string &sessionId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `HallOcrPolicy::OcrFold parking::HallOcrPolicy::onOcrResult(const std::string &sessionId, CaptureStage stage, bool recognized)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `HallOcrPolicy::StageState & parking::HallOcrPolicy::stageRef(SessionState &session, CaptureStage stage)` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `ImageAction parking::HallOcrPolicy::onImageArrived(const std::string &sessionId, CaptureStage stage)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `OcrSessionState parking::HallOcrPolicy::state(const std::string &sessionId) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::HallOcrPolicy::HallOcrPolicy(int maxAttempts=2)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t parking::HallOcrPolicy::trackedSessions()` `const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::HallOcrPolicy::openSession(const std::string &sessionId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/ParkingCorrelation.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `bool parking::BestShotAttachResult::accepted()` `const` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/ParkingOccupancyConfirmationGate.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `ParkingOccupancyConfirmationGate::Decision parking::ParkingOccupancyConfirmationGate::evaluate(const ParkingSensorEvent &event, bool slotAlreadyOccupied)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::ParkingOccupancyConfirmationGate::ParkingOccupancyConfirmationGate(std::chrono::milliseconds confirmThreshold)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< std::chrono::steady_clock::time_point > parking::ParkingOccupancyConfirmationGate::nextDeadline()` `const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< ParkingSensorEvent > parking::ParkingOccupancyConfirmationGate::takeDue(std::chrono::steady_clock::time_point monotonicNow, std::chrono::system_clock::time_point wallNow)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/ParkingOccupancySession.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `ParkingSessionState parking::ParkingOccupancySession::state()` `const` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool parking::ParkingOccupancySession::active()` `const` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

```

const std::optional< std::chrono::system_clock::time_point > &
• parking::ParkingOccupancySession::endedAt() const noexcept
  — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• const std::string & parking::ParkingOccupancySession::sensorId() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• const std::string & parking::ParkingOccupancySession::sessionId() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• const std::string & parking::ParkingOccupancySession::slotId() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
parking::ParkingOccupancySession::ParkingOccupancySession(std::string sessionId, std::string slotId, std::string sensorId, std::chrono::system_clock::time_point startedAt,
• std::chrono::steady_clock::time_point startedAtMonotonic)
  — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
std::chrono::steady_clock::time_point parking::ParkingOccupancySession::startedAtMonotonic() const
• noexcept
  — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• std::chrono::system_clock::time_point parking::ParkingOccupancySession::startedAt() const noexcept
  — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• void parking::ParkingOccupancySession::complete(std::chrono::system_clock::time_point endedAt) — 활성 세션을 완료하며 시작보다 이른 종료 시각은 허용하지 않는다.

```

include/parking/ParkingSensorEvent.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

```

• const char * parking::toString(ParkingSensorState state) noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

```

include/parking/ParkingSensorSequenceGuard.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

```

bool parking::ParkingSensorSequenceGuard::accept(const ParkingSensorEvent &event, std::string
• *reason=nullptr)
  — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• void parking::ParkingSensorSequenceGuard::clear() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• void parking::ParkingSensorSequenceGuard::reset(const std::string &sensorId) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

```

include/parking/ParkingSessionWorker.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

```

parking::ParkingSessionWorker::ParkingSessionWorker(std::vector< ParkingSlotConfig > slotConfigs,
• std::chrono::milliseconds confirmThreshold=std::chrono::milliseconds::zero())
  — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
std::optional< ParkingTransitionResult > parking::ParkingSessionWorker::onSensorEvent(const
• ParkingSensorEvent &event)
  — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• std::size_t parking::ParkingSessionWorker::slotCount() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• void parking::ParkingSessionWorker::addSink(TransitionSink sink) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

```


include/parking/ParkingSlot.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `bool parking::ParkingSlot::occupied() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::ParkingSlot::ParkingSlot(ParkingSlotConfig value)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/ParkingSlotConfig.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `std::vector< ParkingSlotConfig > parking::ParkingSlotConfigLoader::loadFromFile(const std::string &path)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< ParkingSlotConfig > parking::ParkingSlotConfigLoader::parse(const std::string &jsonText)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/ParkingSlotManager.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `ParkingTransitionResult parking::ParkingSlotManager::handle(const ParkingSensorEvent &event)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool parking::ParkingTransitionResult::changed() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const ParkingSlot * parking::ParkingSlotManager::findSlot(const std::string &slotId) const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::ParkingSlotManager::ParkingSlotManager(std::vector< ParkingSlotConfig > configs)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t parking::ParkingSlotManager::activeSlotCount() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t parking::ParkingSlotManager::slotCount() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string parking::ParkingSlotManager::createSessionId(const std::string &slotId,`
`std::chrono::system_clock::time_point startedAt)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/ParkingTriggerCoordinator.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `BestShotAttachResult parking::ParkingTriggerCoordinator::attachBestShotIfActive(const CommittedCorrelationLease &lease, BestShotEvidenceKind kind, const std::string &image_ref, const std::string &image_path, const std::string &plate_text={})` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `ParkingCorrelationMatch parking::ParkingTriggerCoordinator::resolve(const std::string &camera_id, const std::string &channel_id, const std::string &object_id) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::ParkingTriggerCoordinator::ParkingTriggerCoordinator(database::EventDatabase &database, int correlation_window_ms=8000, Clock clock={})` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::chrono::milliseconds parking::ParkingTriggerCoordinator::correlationWindow() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `std::int64_t parking::ParkingTriggerCoordinator::nowEpochMs()` `const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::int64_t parking::ParkingTriggerCoordinator::systemNowEpochMs()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/PlateIlluminator.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `PlateIlluminator::Scope & parking::PlateIlluminator::Scope::operator=(Scope &&other)` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `PlateIlluminator::Scope parking::PlateIlluminator::illuminate(const CaptureRequest &request)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`PlateIlluminator::Scope parking::PlateIlluminator::illuminate(const CaptureRequest &request,`
- `std::chrono::system_clock::time_point now)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `Scope & parking::PlateIlluminator::Scope::operator=(const Scope &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool parking::PlateIlluminator::Scope::lit()` `const` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`bool parking::PlateIlluminator::needsLight(const CaptureRequest &request,`
- `std::chrono::system_clock::time_point now)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`bool parking::PlateIlluminator::send(const std::string &sensorId, const char *state, std::uint32_t`
- `sequence)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::PlateIlluminator::PlateIlluminator(PlateIlluminatorConfig config, LedCommandSender sender)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::PlateIlluminator::Scope::Scope()` `noexcept=default` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`parking::PlateIlluminator::Scope::Scope(PlateIlluminator &owner, std::string sensorId,`
- `std::uint32_t sequence)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::PlateIlluminator::Scope::Scope(Scope &&other)` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::PlateIlluminator::Scope::Scope(const Scope &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::PlateIlluminator::Scope::~Scope()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::uint32_t parking::PlateIlluminator::Scope::sequence()` `const` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::PlateIlluminator::Scope::release()` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::PlateIlluminator::forget(const std::string &sessionId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::PlateIlluminator::forget(std::int64_t sessionId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::PlateIlluminator::markPlateUnreadable(std::int64_t sessionId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::PlateIlluminator::markResolved(std::int64_t sessionId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::PlateIlluminator::note(std::int64_t sessionId, SessionOcr state)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/SensorSlotIndex.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- parking::SensorSlotIndex::SensorSlotIndex(const std::vector< ParkingSlotConfig > &configs) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
std::optional< SensorSlotMatch > parking::SensorSlotIndex::findBySensorId(const std::string &sensorId) const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::size_t parking::SensorSlotIndex::size() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/SlotTransitionActor.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- SlotSubmitResult parking::SlotTransitionActor::submit(const SlotTransitionCommand &command) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- SlotTransitionActor & parking::SlotTransitionActor::operator=(const SlotTransitionActor &)=delete — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotSubmitResult::accepted() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotTransitionActor::ingressOpen() const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotTransitionActor::processDueDeadlines(std::int64_t nowEpochMs) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotTransitionActor::processEffects(std::int64_t nowEpochMs) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotTransitionActor::processIngress(std::int64_t nowEpochMs) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotTransitionActor::processRunnable(std::int64_t nowEpochMs) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotTransitionActor::start() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotTransitionActor::stopAndDrain(std::chrono::milliseconds timeout) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotTransitionActor::waitUntilIdle(std::chrono::milliseconds timeout) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- parking::SlotTransitionActor::SlotTransitionActor(const SlotTransitionActor &)=delete — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
parking::SlotTransitionActor::SlotTransitionActor(database::SessionTransitionStore &store, Config config, EffectSink effectSink) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- parking::SlotTransitionActor::~SlotTransitionActor() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::int64_t parking::SlotTransitionActor::nowEpochMs() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- void parking::SlotTransitionActor::closeIngress() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- void parking::SlotTransitionActor::run() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking/SlotTransitionTypes.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- bool parking::CommittedOccupancyTransition::terminal() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotAdmissionResult::accepted() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/parking_timer

include/parking_timer/EventDatabase.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/parking_timer/EventManager.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `bool parking_timer::EventManager::publish(std::string_view event_type, std::string_view slot_id, std::string_view car_number, std::string_view occurred_at, std::string_view detail={}, std::int64_t session_id=-1)`
— slot_id, 차량, 시각과 상세 근거를 JSON 이벤트로 출력한다.
- `void parking_timer::EventManager::setPublisher(Publisher publisher)` — JSON 로그 외에 MQTT 등 외부 전달 callback을 설정한다.

include/parking_timer/ParkingSlotManager.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `EntryResult parking_timer::ParkingSlotManager::handleEntry(const std::string &slot_id, const std::string &car_number, const std::string &image_path_1={})`
— EV/PHEV 입차만 세션과 타이머로 등록하고 중복 입차를 거부한다.
- `EntryResult parking_timer::ParkingSlotManager::handleRecognizedSession(std::int64_t session_id, const std::string &slot_id, const std::string &car_number)`
— 카메라 흐름이 이미 만든 세션을 중복 INSERT 없이 타이머에 등록한다.
- `bool parking_timer::ParkingSlotManager::start() noexcept` — Starts the owned timer worker after runtime teardown is armed.
- `parking_timer::ParkingSlotManager::ParkingSlotManager(EventDatabase &database, EventManager &events, std::chrono::milliseconds parking_timeout, TimerManager::EvidenceProvider &evidence_provider={})`
— 입·출차 상태 전이와 EV 점유 타이머를 조정하는 관리자를 생성한다.
- `std::chrono::milliseconds parking_timer::ParkingSlotManager::parkingTimeout() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< LogRecord > parking_timer::ParkingSlotManager::handleCommittedExit(std::int64_t session_id, const std::string &slot_id)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< LogRecord > parking_timer::ParkingSlotManager::handleExit(const std::string &slot_id)`
— 활성 세션을 출차 처리하며 없으면 nullopt를 반환한다.
- `std::size_t parking_timer::ParkingSlotManager::pendingTimerCount() const` — lazy-canceled 항목을 포함한 현재 우선순위 큐 크기를 반환한다.
- `std::size_t parking_timer::ParkingSlotManager::restoreActiveSessions()` — 서버 재시작 시 DB의 EV/PHEV 활성 세션을 타이머 큐에 복구한다.
- `std::size_t parking_timer::ParkingSlotManager::updateParkingTimeout(std::chrono::milliseconds parking_timeout)`
— 활성 EV/PHEV 세션을 원래 T0 기준 새 제한시간으로 모두 재예약한다.

include/parking_timer/RuntimeConfig.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `RuntimeConfig parking_timer::RuntimeConfig::load(const std::filesystem::path &file)` — KEY=VALUE 형식의 설정 파일을 읽어 런타임 설정을 만든다.

- `void parking_timer::RuntimeConfig::applyEnvironment()` — 지원하는 환경변수로 현재 런타임 설정을 덮어쓴다.

include/parking_timer/TimerManager.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `TimerManager & parking_timer::TimerManager::operator=(const TimerManager &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool parking_timer::TimerManager::LaterDeadline::operator()(const TimerItem &left, const TimerItem &right) const noexcept` — `priority_queue` 에서 더 늦은 항목의 우선순위를 낮추는 비교 연산자.
- `bool parking_timer::TimerManager::isCurrent(const TimerItem &item) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool parking_timer::TimerManager::start() noexcept` — Starts the deadline worker after the runtime teardown guard exists.
- `parking_timer::TimerManager::TimerManager(EventDatabase &database, ViolationCallback callback, ErrorCallback error_callback={}, std::mutex *transition_mutex=nullptr, EvidenceProvider evidence_provider={})` — DB와 callback을 연결하되 deadline worker는 아직 시작하지 않는다.
- `parking_timer::TimerManager::TimerManager(const TimerManager &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking_timer::TimerManager::~~TimerManager()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t parking_timer::TimerManager::pendingCount() const` — 아직 worker가 소비하지 않은 큐 항목 수를 반환한다.
- `void parking_timer::TimerManager::processExpired(TimerItem item)` — 만료 노드를 DB 조건부 UPDATE로 검증하고 위반 callback을 발생시킨다.
- `void parking_timer::TimerManager::reportError(const TimerItem &item, std::string message) noexcept` — 타이머 오류를 등록된 callback 또는 표준 오류 출력으로 안전하게 보고한다.
- `void parking_timer::TimerManager::reschedule(std::int64_t log_id, std::string slot_id, std::string car_number, std::chrono::milliseconds delay)` — 기존 세션 deadline을 무효화하고 새 기준시간으로 교체한다.
- `void parking_timer::TimerManager::retryAfterDatabaseError(TimerItem item, std::string message) noexcept` — 일시적인 SQLite 오류가 난 타이머를 지수 backoff로 다시 예약한다.
- `void parking_timer::TimerManager::retryAfterEvidencePending(TimerItem item) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking_timer::TimerManager::run()` — 가장 이른 deadline만 기다리며 만료 노드를 처리하는 worker 루프.
- `void parking_timer::TimerManager::schedule(std::int64_t log_id, std::string slot_id, std::string car_number, std::chrono::milliseconds delay)` — 불변 session ID의 위반 deadline을 큐에 등록하고 worker를 깨운다.
- `void parking_timer::TimerManager::scheduleImpl(std::int64_t log_id, std::string slot_id, std::string car_number, std::chrono::milliseconds delay)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking_timer::TimerManager::stop() noexcept` — Stops and joins the deadline worker. Safe to call repeatedly.

include/parking_timer/Types.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/sensor

include/sensor/FireSensorMessage.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/sensor/HallParkingService.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `HallParkingService & sensor::HallParkingService::operator=(const HallParkingService &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `HallParkingWorkQueue::PushResult sensor::HallParkingWorkQueue::push(HallParkingWorkItem item)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`bool sensor::HallParkingService::applyCommittedEffects(const parking::CommittedOccupancyTransition &transition)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool sensor::HallParkingService::handleCameraIvaSignal(const event::IvaOccupancySignal &signal)` — WiseAI IVA 액션을 기존 세션·정리 흐름으로 연결한다.
`bool sensor::HallParkingService::handleLine(const std::string &line, const std::string &transport="mqtt-test")` — SENSOR:HALLxx:OCCUPIED/VACANT 메시지 한 줄을 처리한다.
- `bool sensor::HallParkingService::removeEarlyDepartureImages(std::int64_t session_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool sensor::HallParkingService::stop(std::chrono::milliseconds timeout=std::chrono::seconds(30))` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool sensor::HallParkingWorkQueue::canAccept(const parking::ParkingSensorEvent &event) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool sensor::HallParkingWorkQueue::empty() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`parking::SlotTransitionCommand sensor::HallParkingService::makeCameraCommand(const event::IvaOccupancySignal &signal) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`parking::SlotTransitionCommand sensor::HallParkingService::makeHallCommand(const parking::ParkingSensorEvent &event, const std::string &raw_line) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `sensor::HallParkingService::HallParkingService(const HallParkingService &)=delete` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`sensor::HallParkingService::HallParkingService(std::vector< parking::ParkingSlotConfig > slot_configs, const app::AppConfig &app_config, std::vector< std::shared_ptr< camera::CameraChannel > > &channels, database::EventDatabase &database, OcrCancel ocr_cancel, parking_timer::ParkingSlotManager &timer_manager, parking_timer::EventManager &event_manager, parking::EvidenceCaptureWorker &evidence_worker, event::SystemEventReporter *system_event_reporter=nullptr, TransitionSink transition_sink={}, std::function< void(parking::SlotActorCheckpoint)> actor_checkpoint={})` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `sensor::HallParkingService::~~HallParkingService()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `sensor::HallParkingWorkQueue::HallParkingWorkQueue(std::size_t capacity)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< HallParkingWorkItem > sensor::HallParkingWorkQueue::pop()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t sensor::HallParkingWorkQueue::capacity() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t sensor::HallParkingWorkQueue::size() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- void sensor::HallParkingService::report(event::SystemEventCode code, event::SystemEventSeverity severity, const std::string &message, const std::string &transport, const std::string &slot_id={})
- noexcept
 - Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/sensor/ParkingSensorEventAdapter.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- sensor::ParkingSensorEventAdapter::ParkingSensorEventAdapter(const parking::SensorSlotIndex &slotIndex)
 - Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::optional< parking::ParkingSensorEvent > sensor::ParkingSensorEventAdapter::adapt(const SensorProtocolMessage &message, std::string *error=nullptr) const
 - 미등록 sensor_id면 nullptr와 선택적 오류 문자열을 반환한다.

include/sensor/SensorProtocolMessage.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/sensor/SensorProtocolParser.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- bool sensor::SensorProtocolParser::isFireLine(const std::string &line) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::optional< FireSensorMessage > sensor::SensorProtocolParser::parseFire(const std::string &line, std::chrono::system_clock::time_point receivedAt, std::string *error=nullptr) const
 - Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::optional< SensorProtocolMessage > sensor::SensorProtocolParser::parse(const std::string &line, std::chrono::system_clock::time_point receivedAt, std::string *error=nullptr) const
 - Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/sensor/SensorProtocolVersion.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- constexpr const char * sensor::toString(const SensorProtocolVersion version) noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::optional< SensorProtocolVersion > sensor::sensorProtocolVersionFromString(const std::string_view value) noexcept
 - Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/sensor/SensorSequencePolicy.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- SensorSequenceDecision sensor::evaluateSensorSequence(const SensorSequenceState &state, const SensorSequenceFact &fact)
 - Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool sensor::SensorSequenceDecision::accepted() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- void sensor::commitSensorSequence(SensorSequenceState &state, const SensorSequenceFact &fact) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/settings

include/settings/OverstayThresholdService.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `OverstayThresholdStatus settings::OverstayThresholdService::status() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `ThresholdUpdateResult settings::OverstayThresholdService::update(int seconds)` — DB 저장 성공 후에만 메모리와 런타임 타이머에 새 값을 적용한다.
- `bool settings::OverstayThresholdService::initialize()` — DB 값을 로드하며 없으면 bootstrap 기본값을 영구 저장한다.
- `bool settings::OverstayThresholdService::isValid(int seconds) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `int settings::OverstayThresholdService::thresholdSeconds() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `settings::OverstayThresholdService::OverstayThresholdService(database::EventDatabase &database, int bootstrap_seconds=kDefaultSeconds)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void settings::OverstayThresholdService::setApplyCallback(ApplyCallback callback)` — 타이머·증거 worker 재예약 callback을 서버 조립 단계에서 주입한다.

include/settings/ParkingRoiSettingsService.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `ParkingRoiUpdateResult settings::ParkingRoiSettingsService::update(const std::string &slot_id, snapshot::NormalizedRoi roi)` — DB 저장 성공 후에만 실행 중 좌표를 즉시 교체한다.
- `bool settings::ParkingRoiSettingsService::initialize()` — DB 저장값을 불러오고, 없으면 AppConfig 좌표를 초기값으로 저장한다.
- `bool settings::ParkingRoiSettingsService::isValid(const snapshot::NormalizedRoi &roi) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `settings::ParkingRoiSettingsService::ParkingRoiSettingsService(database::EventDatabase &database, const std::vector< app::IvaAreaConfig > &bootstrap)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< parking::AppliedParkingRoi > settings::ParkingRoiSettingsService::parse(const std::string &slot_id, const std::string &value)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< parking::AppliedParkingRoi > settings::ParkingRoiSettingsService::resolveForUse(const std::string &slot_id) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< snapshot::NormalizedRoi > settings::ParkingRoiSettingsService::roiForSlot(const std::string &slot_id) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string settings::ParkingRoiSettingsService::serialize(const parking::AppliedParkingRoi &roi)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string settings::ParkingRoiSettingsService::settingKey(const std::string &slot_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< ParkingRoiSetting > settings::ParkingRoiSettingsService::list() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/snapshot

include/snapshot/NormalizedRoi.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/snapshot/SnapshotStorage.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- `StoredImagePair snapshot::SnapshotStorage::saveCameraApiHallCapture(const std::string &channel_id, std::int64_t session_id, const std::string &slot_id, const std::string &capture_stage, const NormalizedRoi &roi, const std::vector< unsigned char > &original_jpeg, const std::vector< unsigned char > &enhanced_jpeg)`
— 카메라 CAP original/enhanced JPEG를 같은 슬롯 ROI로 잘라 저장한다.
- `StoredImagePair snapshot::SnapshotStorage::saveCameraApiIvaSnapshot(const std::string &channel_id, const std::string &slot_id, const NormalizedRoi &roi, const std::vector< unsigned char > &original_jpeg, const std::vector< unsigned char > &enhanced_jpeg)`
— 세션이 없는 IVA 이벤트의 original/enhanced JPEG를 ROI로 잘라 저장한다.
- `cv::Mat snapshot::SnapshotStorage::waitForFullFrame(const std::shared_ptr< camera::CameraChannel > &channel)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `snapshot::SnapshotStorage::SnapshotStorage(const std::string &snapshot_dir, int snapshot_frame_wait_ms, std::atomic< bool > &running)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string snapshot::SnapshotStorage::saveAreaSnapshot(const std::shared_ptr< camera::CameraChannel > &channel, const std::string &slot_id, const NormalizedRoi &roi, const std::string &filename_prefix, std::int64_t session_id=-1, const std::string &stage_directory={})`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string snapshot::SnapshotStorage::saveEvidenceSnapshot(const std::shared_ptr< camera::CameraChannel > &channel, std::int64_t session_id, const std::string &slot_id, const std::string &evidence_reason, const NormalizedRoi &roi)`
— 세션·증거 종류가 포함된 이름으로 최신 ROI 원본을 저장한다.
- `std::string snapshot::SnapshotStorage::saveFullSizeSnapshot(const std::shared_ptr< camera::CameraChannel > &channel)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string snapshot::SnapshotStorage::saveHallCaptureSnapshot(const std::shared_ptr< camera::CameraChannel > &channel, std::int64_t session_id, const std::string &slot_id, const std::string &capture_stage, const NormalizedRoi &roi)`
— 30/60초 홀 촬영본을 실제 DB 세션 ID가 포함된 이름으로 저장한다.
- `std::string snapshot::SnapshotStorage::saveIvaAreaSnapshot(const std::shared_ptr< camera::CameraChannel > &channel, const std::string &slot_id, const NormalizedRoi &roi)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

include/uapi

include/uapi/parking_alert.h

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/util

include/util/Logger.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/util/StringUtil.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/util/TimeUtil.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

include/util/UrlMasker.hpp

공개 인터페이스, 타입 또는 클래스 선언을 정의한다.

- 함수 없음: 타입·상수·구조체 선언 또는 데이터 전용 파일

src/app

src/app/AppConfig.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- AppConfig app::AppConfig::loadFromEnv() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/app/RuntimeShutdown.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- app::RuntimeShutdown::RuntimeShutdown(RuntimeShutdownHooks hooks) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- app::RuntimeShutdown::~~RuntimeShutdown() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool app::RuntimeShutdown::failed() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool app::RuntimeShutdown::shutdown() noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool app::RuntimeShutdown::stopped() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/bestshot

src/bestshot/BestShotReceiver.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- BestShotProcessResult bestshot::BestShotReceiver::processOne(PendingMetadata metadata, bool may_requeue)
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bestshot::BestShotReceiver::BestShotReceiver(std::vector< std::shared_ptr< camera::CameraChannel > > &channels, parking::ParkingTriggerCoordinator &trigger_coordinator, ocr::OcrWorker &ocr_worker, std::atomic< bool > &running, std::string output_root="data/bestshots", DownloadCallback downloader={}, OcrCallback ocr_callback={}, std::size_t pending_capacity=64)
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bestshot::BestShotReceiver::~~BestShotReceiver() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool bestshot::BestShotReceiver::downloadImage(const std::string &rtsp_url, const std::string &image_ref, const std::string &destination)
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool bestshot::BestShotReceiver::enqueuePending(PendingMetadata metadata) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::string bestshot::BestShotReceiver::evidenceIdentity(const PendingMetadata &metadata) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `std::string bestshot::BestShotReceiver::exactIdentityKey(const PendingMetadata &metadata)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string bestshot::BestShotReceiver::safePathComponent(const std::string &value)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string bestshot::BestShotReceiver::stableDigest(const std::string &value)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`std::vector< BestShotProcessResult > bestshot::BestShotReceiver::processMetadataDocument(const std::string &camera_id, const std::string &channel_id, const std::string &xml, const std::string &rtsp_url)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< BestShotProcessResult > bestshot::BestShotReceiver::reconcilePending()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void bestshot::BestShotReceiver::pendingLoop()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`void bestshot::BestShotReceiver::receiveLoop(const std::shared_ptr< camera::CameraChannel > &channel)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void bestshot::BestShotReceiver::start()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void bestshot::BestShotReceiver::stop()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/camera

src/camera/CameraSnapshotApiClient.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `CameraSnapshotApiClient::HttpResponse camera::CameraSnapshotApiClient::request(const std::string &method, const std::string &url, const std::string &jsonBody, int timeoutMs) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool camera::CameraSnapshotApiClient::configured() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool camera::CameraSnapshotApiClient::discoverChannels()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool camera::CameraSnapshotApiClient::discoverFilters()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`bool camera::CameraSnapshotApiClient::downloadJpeg(const std::string &path, std::size_t expectedBytes, std::vector< unsigned char > &output)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool camera::CameraSnapshotApiClient::generate(int channel, CameraGeneratedImages &images)` — 같은 카메라 프레임의 original/enhanced JPEG를 생성·다운로드한다.
- `bool camera::CameraSnapshotApiClient::initialize()` — 이미지 서버 시작과 channels/filters discovery를 수행한다.
- `bool camera::CameraSnapshotApiClient::initializeUnlocked()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool camera::CameraSnapshotApiClient::startImageServer()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `camera::CameraSnapshotApiClient::CameraSnapshotApiClient(CameraSnapshotApiConfig config)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const std::string & camera::CameraSnapshotApiClient::lastError() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const std::vector< int > & camera::CameraSnapshotApiClient::channels() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const std::vector< std::string > & camera::CameraSnapshotApiClient::filters() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void camera::CameraSnapshotApiClient::setError(std::string message)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/camera/RtspStreamReceiver.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `bool camera::RtspStreamReceiver::waitForInitialFrames(int timeout_sec)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`camera::RtspStreamReceiver::RtspStreamReceiver(std::vector< std::shared_ptr< CameraChannel > > &channels, int preview_width, int preview_height, int rtsp_retry_delay_ms, int`
- `empty_frame_delay_ms, int max_consecutive_read_failures, std::atomic< bool > &running)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void camera::RtspStreamReceiver::captureLoop(std::shared_ptr< CameraChannel > channel)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void camera::RtspStreamReceiver::start()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void camera::RtspStreamReceiver::stop()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/database

src/database/EventDatabase.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `bool database::EventDatabase::attachEnhancedPlateImage(const std::string &image_path, const std::string &enhanced_image_path)` — 원본 IMAGE_LOG 행에 OpenCV 전처리 파일 경로를 연결한다.
- `bool database::EventDatabase::attachPlateBestShot(int session_id, const std::string &image_path, const std::string &plate_text)` — 번호판 BestShot과 선택적 카메라 plate text를 기존 세션에 연결한다.
- `bool database::EventDatabase::attachVehicleBestShot(int session_id, const std::string &image_path, const std::string &object_id)` — 이미 활성 세션이 있는 슬롯에 차량 BestShot 이미지만 연결한다(세션/슬롯상태는 건드리지 않음).
- `bool database::EventDatabase::createEntryWithBestShot(const std::string &slot_id, const std::string &image_path, const std::string &object_id, int *session_id)` — Vehicle BestShot을 근거로 OCCUPIED 슬롯과 ACTIVE 세션을 원자적으로 연결한다.
- `bool database::EventDatabase::createEntryWithSnapshot(const std::string &slot_id, const std::string &image_path, const std::string &source_id, int *session_id)` — 홀센서 입차 Snapshot으로 ACTIVE 세션과 IMAGE/EVENT 로그를 만든다.
- `bool database::EventDatabase::deleteSessionImageRecords(int session_id)` — 파일 삭제가 끝난 조기 출차 세션의 IMAGE_LOG 행을 모두 제거한다.
- `bool database::EventDatabase::getImage(int image_id, ImageView &row)` — image_id로 이미지 경로와 OCR 메타데이터를 조회한다.
- `bool database::EventDatabase::getParkingSlot(const std::string &slot_id, ParkingSlotView &row)` — slot_id 한 건의 상태를 조회한다.
- `bool database::EventDatabase::insertEvent(const EventRecord &record)` — 정규화된 카메라 이벤트와 선택적 Snapshot을 IMAGE_LOG/EVENT_LOG에 기록한다.
- `bool database::EventDatabase::insertSystemEvent(const std::string &event_type, const std::string &slot_id, const std::string &message)` — 센서·통신 운영 이벤트를 기존 EVENT_LOG schema에 저장한다.
- `bool database::EventDatabase::listParkingSlots(std::vector< ParkingSlotView > &rows)` — 전체 주차면과 활성 세션을 조회한다.
- `bool database::EventDatabase::listSessionImages(int session_id, std::vector< ImageView > &rows)` — 세션에 연결된 모든 이미지 메타데이터를 조회한다.
- `bool database::EventDatabase::occupancySchemaReady()` `const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::EventDatabase::open(const std::string &db_path)` — SQLite 파일을 열고 FK 검사를 활성화한다.

- `bool database::EventDatabase::runtimeSchemaReady() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `database::EventDatabase::EventDatabase()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `database::EventDatabase::~EventDatabase()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string database::EventDatabase::applyPlateOcr(int session_id, const std::string &slot_id, const std::string &image_path, const std::string &plate_number, double confidence)` — OCR 결과를 저장하고 VEHICLE 조회 결과(EV/NON_EV/UNKNOWN)를 반환한다.
- `void database::EventDatabase::close()` — 열린 DB 연결을 닫는다.

src/database/EventDatabaseCorrelation.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `parking::BestShotAttachResult database::EventDatabase::attachBestShotIfActive(const parking::CommittedCorrelationLease &lease, parking::BestShotEvidenceKind kind, const std::string &image_ref, const std::string &image_path, const std::string &plate_text, std::int64_t now_epoch_ms)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::ParkingCorrelationMatch database::EventDatabase::resolveParkingCorrelation(const std::string &camera_id, const std::string &channel_id, const std::string &object_id, std::int64_t now_epoch_ms) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/database/EventDatabaseFire.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `bool database::EventDatabase::acknowledgeFireDelivery(const std::string &delivery_key, std::uint64_t fire_revision)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::EventDatabase::isSensorBootIdRetired(const std::string &source_kind, const std::string &sensor_id, const std::string &boot_id) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::EventDatabase::markFireDeliveryInFlight(const std::string &delivery_key, std::uint64_t fire_revision)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::EventDatabase::markFireDeliveryPending(const std::string &delivery_key, std::uint64_t fire_revision, const std::string &error)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::EventDatabase::resetFireInFlightDeliveries()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `event::FireStoreMutationResult database::EventDatabase::applyFireStateMutation(const event::FireStateMutation &mutation)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `event::FireStoreMutationResult database::EventDatabase::initializeFireTopology(const std::vector< event::FireChannelBootstrap > &topology)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< event::FireAlarmStateRecord > database::EventDatabase::getFireAlarmState(const std::string &channel_id) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< event::FireOutboxRecord > database::EventDatabase::getFireDelivery(const std::string &delivery_key) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< event::FireAlarmStateRecord > database::EventDatabase::listFireAlarmStates() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `std::vector< event::FireOutboxRecord > database::EventDatabase::listFireLifecycleDeliveries() const`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< event::FireOutboxRecord > database::EventDatabase::listFireRetainedDeliveries() const`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< event::FireOutboxRecord >`
`database::EventDatabase::listPendingFireLifecycleDeliveries(const std::optional< std::string >`
`&channel_id=std::nullopt) const`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/database/EventDatabaseOccupancy.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `bool database::EventDatabase::completeSlotTransitionEffects(const std::string &command_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::EventDatabase::deferSlotTransitionCommand(const std::string &command_id,`
`std::int64_t next_attempt_at_epoch_ms, const std::string &error) noexcept`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::EventDatabase::deferSlotTransitionEffects(const std::string &command_id,`
`std::int64_t next_attempt_at_epoch_ms, const std::string &error) noexcept`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::CommittedOccupancyTransition database::EventDatabase::applySlotTransitionCommand(const`
`std::string &command_id)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::SlotAdmissionResult database::EventDatabase::admitSlotTransitionCommand(const`
`parking::SlotTransitionCommand &command, std::size_t pending_capacity)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< std::int64_t > database::EventDatabase::nextScheduledSlotDeadlineEpochMs() const` —
Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t database::EventDatabase::admitDueSlotDeadlines(std::int64_t now_epoch_ms, std::size_t`
`pending_capacity, const std::vector< std::string > &blocked_slots={})`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t database::EventDatabase::pendingSlotTransitionCommandCount() const` — Doxygen 설명이 없어 선언
과 호출부를 함께 확인해야 한다.
- `std::size_t database::EventDatabase::pendingSlotTransitionDrainCount(std::int64_t`
`shutdown_cutoff_epoch_ms) const`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t database::EventDatabase::pendingSlotTransitionEffectCount() const` — Doxygen 설명이 없어 선언과
호출부를 함께 확인해야 한다.
- `std::vector< parking::CommittedOccupancyTransition >`
`database::EventDatabase::listPendingSlotTransitionEffects(std::int64_t now_epoch_ms) const`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< parking::DurableSlotCommand >`
`database::EventDatabase::listRunnableSlotTransitionCommands(std::int64_t now_epoch_ms) const`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/database/EventDatabaseTimer.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `EvidenceInsertResult database::EventDatabase::insertEvidenceImage(std::int64_t session_id, const`
`std::string &original_path, const std::string &evidence_reason, const std::string &captured_at,`
`const std::string &enhanced_path={}, snapshot::NormalizedRoi applied_roi={}, std::uint64_t`
`roi_revision=0)`
— 활성 세션에 종류별 증거 이미지 한 장만 원자적으로 연결한다.

```

EvidenceInsertResult database::EventDatabase::insertHallCaptureImage(std::int64_t session_id, const
std::string &original_path, const std::string &enhanced_path, const std::string &enhancement_type,
const std::string &captured_at, snapshot::NormalizedRoi applied_roi={}, std::uint64_t
• roi_revision=0)
— 활성 세션에 30/60초 ROI 촬영본을 단계별 최대 한 장 연결한다.
• VehicleCategory database::EventDatabase::classifyVehicle(std::string_view car_number) const —
VEHICLE의 is_ev/is_phev로 차량 종류를 분류한다.
bool database::EventDatabase::cancelUnscheduled(std::int64_t log_id, const std::string
• &canceled_at)
— DB 생성 뒤 timer enqueue 실패 시 세션을 보상 종료한다.
bool database::EventDatabase::markPlateOcrUnresolved(std::int64_t session_id, const std::string
• &slot_id, int attempts)
— OCR 시도 소진을 UNKNOWN으로 한 번만 EVENT_LOG에 기록한다.
bool database::EventDatabase::markViolation(std::int64_t log_id, const std::string &violation_at,
• const std::string &image_path_2)
— 아직 ACTIVE인 세션만 VIOLATION으로 조건부 갱신한다.
• bool database::EventDatabase::upsertSystemSetting(const std::string &key, const std::string &value)
— 런타임 설정을 원자적으로 추가하거나 갱신한다.
• database::EventDatabase::EventDatabase(const std::filesystem::path &database_path) — SQLite 이벤트 DB를
열고 프로토타입에 필요한 연결 옵션을 설정한다.
std::int64_t database::EventDatabase::createHallSession(const std::string &slot_id, const
• std::string &source_id, const std::string &entry_time)
— 홀센서 입차의 ACTIVE 세션을 만들고 실제 SQLite ID를 반환한다.
std::int64_t database::EventDatabase::insertParked(const std::string &car_number, const std::string
• &slot_id, const std::string &parked_at, const std::string &image_path_1)
— EV/PHEV 장기 점유용 ACTIVE 세션과 최초 이미지를 트랜잭션으로 생성한다.
std::optional< LogRecord > database::EventDatabase::departActiveBySlot(const std::string &slot_id,
• const std::string &departed_at)
— slot_id의 활성 세션을 ENDED로 바꾸고 점유시간을 계산한다.
std::optional< LogRecord > database::EventDatabase::findActiveBySlot(const std::string &slot_id)
• const
— 주차면의 출차되지 않은 최신 세션을 조회한다.
• std::optional< LogRecord > database::EventDatabase::findLogById(std::int64_t log_id) const — 불변
session ID로 타이머 읽기 모델을 조회한다.
std::optional< std::string > database::EventDatabase::findEvidenceImagePath(std::int64_t
• session_id, const std::string &evidence_reason) const
— 이미 저장된 세션 증거 이미지 경로를 조회한다.
std::optional< std::string > database::EventDatabase::getSystemSetting(const std::string &key)
• const
— 런타임 설정 문자열을 조회한다. 키가 없으면 nullptr를 반환한다.
• std::string database::EventDatabase::readTextFile(const std::filesystem::path &path) — SQL/config 보조
파일 전체를 문자열로 읽는다.
• std::vector< LogRecord > database::EventDatabase::listLogs() const — 타이머 CLI 표시용 전체 세션을 생성 순서로
반환한다.
• std::vector< std::pair< std::string, std::string > > database::EventDatabase::listVehicles() const
— 차량번호와 EV/PHEV/NON_EV 문자열 목록을 반환한다.
• void database::EventDatabase::clearTimerLogs() — TIMER_ENTRY로 식별되는 데모 타이머 세션만 정리한다.
• void database::EventDatabase::executeSqlUnlocked(const std::string &sql) — 준비 과정이 필요 없는 SQL 문자열
을 SQLite 연결에서 직접 실행한다.
void database::EventDatabase::initialize(const std::filesystem::path &schema_file, const
• std::filesystem::path &seed_file)
— schema와 seed SQL을 적용하며 구형 컬럼을 먼저 호환 마이그레이션한다.
• void database::EventDatabase::migrateRuntimeSchema() — 서버 시작 시 운영 DB에 안전한 맥등 migration만 적용한다.

```

src/database/SessionTransitionStore.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `bool database::SessionTransitionStore::completeEffects(const std::string &commandId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`bool database::SessionTransitionStore::defer(const std::string &commandId, std::int64_t`
- `nextAttemptAtEpochMs, const std::string &error)` noexcept
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`bool database::SessionTransitionStore::deferEffects(const std::string &commandId, std::int64_t`
- `nextAttemptAtEpochMs, const std::string &error)` noexcept
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool database::SessionTransitionStore::ready() const` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `database::SessionTransitionStore::SessionTransitionStore(EventDatabase &database)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`parking::CommittedOccupancyTransition database::SessionTransitionStore::apply(const std::string`
- `&commandId)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`parking::SlotAdmissionResult database::SessionTransitionStore::admit(const`
- `parking::SlotTransitionCommand &command, std::size_t pendingCapacity)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< std::int64_t > database::SessionTransitionStore::nextDeadlineEpochMs() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`std::size_t database::SessionTransitionStore::admitDueDeadlines(std::int64_t nowEpochMs,`
- `std::size_t pendingCapacity, const std::vector< std::string > &blockedSlots={})`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t database::SessionTransitionStore::pendingCount() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`std::size_t database::SessionTransitionStore::pendingDrainCount(std::int64_t shutdownCutoffEpochMs)`
- `const`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t database::SessionTransitionStore::pendingEffectCount() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`std::vector< parking::CommittedOccupancyTransition >`
- `database::SessionTransitionStore::listPendingEffects(std::int64_t nowEpochMs) const`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`std::vector< parking::DurableSlotCommand >`
- `database::SessionTransitionStore::listRunnable(std::int64_t nowEpochMs) const`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/database/db_manager.c

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `int db_assign_vehicle_to_session(int session_id, int vehicle_id, const char *plate_number)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`int db_create_parking_session(int vehicle_id, const char *slot_id, const char *plate_number, int`
- `*session_id)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `int db_delete_session_images(int session_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `int db_end_parking_session(int session_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `int db_get_image_by_id(int image_id, DbImageRow *row)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `int db_get_vehicle_by_plate(const char *plate_number, int *vehicle_id, int *is_ev, int *is_phev)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `int db_insert_event_log(int session_id, const char *slot_id, const char *event_type, const char *message)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `int db_insert_image_log(int session_id, const char *original_path, const char *enhanced_path, const char *enhancement_type, const char *ocr_result)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `int db_insert_image_log_with_roi(int session_id, const char *original_path, const char *enhanced_path, const char *enhancement_type, const char *ocr_result, double roi_x, double roi_y, double roi_width, double roi_height, unsigned long long roi_revision)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `int db_mark_event_handled(int event_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `int db_open(const char *path)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `int db_update_image_enhanced_by_path(const char *original_path, const char *enhanced_path)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `int db_update_image_ocr_by_path(const char *original_path, const char *ocr_result)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `int db_update_slot_status(const char *slot_id, const char *status)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `int db_visit_parking_slots(const char *slot_id, DbParkingSlotVisitor visitor, void *context)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `int db_visit_session_images(int session_id, DbImageVisitor visitor, void *context)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `sqlite3 * db_native_handle(void)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static int bind_id_or_null(sqlite3_stmt *stmt, int index, int value)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static int bind_text_or_null(sqlite3_stmt *stmt, int index, const char *value)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static int finish_update(sqlite3_stmt *stmt, const char *context, int require_change)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static int prepare(sqlite3_stmt **stmt, const char *sql, const char *context)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static int require_db(const char *context)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static void copy_column_text(sqlite3_stmt *stmt, int column, char *destination, size_t capacity)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static void fill_image_row(sqlite3_stmt *stmt, DbImageRow *row)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `struct sqlite3 * db_native_handle(void)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void db_close(void)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/device

src/device/LinuxDriverAdapter.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `LinuxDriverAdapter & device::LinuxDriverAdapter::operator=(LinuxDriverAdapter &&other)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `ParkingAlertState device::LinuxDriverAdapter::state() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool device::LinuxDriverAdapter::isOpen() const` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const std::string & device::LinuxDriverAdapter::devicePath() const` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `device::LinuxDriverAdapter::LinuxDriverAdapter(LinuxDriverAdapter &&other)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `device::LinuxDriverAdapter::LinuxDriverAdapter(std::string devicePath="/dev/parking_alert")` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `device::LinuxDriverAdapter::~LinuxDriverAdapter()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::LinuxDriverAdapter::clearAll()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::LinuxDriverAdapter::clearSlot(std::uint32_t slotIndex, std::uint64_t eventId=0)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::LinuxDriverAdapter::closeDevice() noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::LinuxDriverAdapter::openDevice()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::LinuxDriverAdapter::requireOpen() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::LinuxDriverAdapter::setSlot(std::uint32_t slotIndex, std::uint64_t eventId=0)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::LinuxDriverAdapter::updateSlot(unsigned long request, std::uint16_t operation, std::uint32_t slotIndex, std::uint64_t eventId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/device/LoRaDriver.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `bool device::LoRaDriver::send(const LoRaFrame &frame, std::string *error=nullptr)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool device::LoRaDriver::sendTo(const LoRaDestination &destination, const LoRaFrame &frame, std::string *error=nullptr)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::uint16_t device::LoRaDriver::crc16Ccitt(std::span< const std::uint8_t > bytes)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< LoRaFrame > device::LoRaDriver::consume(std::span< const std::uint8_t > bytes)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< std::uint8_t > device::LoRaDriver::encode(const LoRaFrame &frame)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/device/SensorLinkManager.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `SensorLinkMode device::SensorLinkManager::parseMode(const std::string &value)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool device::SensorLinkManager::connected() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool device::SensorLinkManager::sendAlertCommand(const std::string &command, std::uint32_t sequence, std::string *error=nullptr)` — STM32/LoRa 반대 방향으로 경고 명령을 전송한다.
- `bool device::SensorLinkManager::start()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool device::SensorLinkManager::waitReconnect()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `device::SensorLinkManager::SensorLinkManager(Config config, SensorLineHandler handler, event::SystemEventReporter *reporter=nullptr)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `device::SensorLinkManager::~SensorLinkManager()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string device::SensorLinkManager::modeName(SensorLinkMode mode)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::SensorLinkManager::consumeLoRaFrames(const std::uint8_t *data, std::size_t size)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::SensorLinkManager::consumeUartLines(const std::uint8_t *data, std::size_t size)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `void device::SensorLinkManager::report(event::SystemEventCode code, event::SystemEventSeverity severity, const std::string &message, std::uint32_t retry_count=0, bool recovered=false) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::SensorLinkManager::run()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void device::SensorLinkManager::stop()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/device/UartDriver.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `bool device::UartDriver::connect(std::string *error=nullptr)` — UART 장치를 non-blocking raw 8N1 모드로 연다.
- `bool device::UartDriver::connected() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool device::UartDriver::writeAll(std::span< const std::uint8_t > data, std::string *error=nullptr)` — 전체 byte가 전송될 때까지 partial write를 처리한다.
- `device::UartDriver::UartDriver(Config config)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `device::UartDriver::~~UartDriver()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `int device::UartDriver::readSome(std::span< std::uint8_t > output, std::string *error=nullptr)` — poll 후 수신 가능한 byte를 읽는다.
- `void device::UartDriver::disconnect()` — 열려 있는 descriptor를 닫는다.

src/event

src/event/CameraEventParser.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `CameraEvent event::CameraEventParser::parse(const std::string &raw_topic, const std::string &raw_payload, const std::string &default_channel_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::CameraEventParser::parseEventChannelId(const std::string &topic, const std::string &default_channel_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::CameraEventParser::parseEventType(const std::string &topic, const std::string &payload)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::CameraEventParser::parseSeverity(const std::string &event_type)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::CameraEventParser::parseSourceId(const std::string &topic)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::CameraEventParser::parseVideoSourceToken(const std::string &topic)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< CameraEvent > event::CameraEventParser::parseMany(const std::string &raw_topic, const std::string &raw_payload, const std::string &default_channel_id)` — 한 MQTT payload의 NotificationMessage들을 개별 이벤트로 분리한다.

src/event/EventPayloadBuilder.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `std::string event::EventPayloadBuilder::buildFireEventId(const FireSignal &signal)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::EventPayloadBuilder::buildFireJson(const std::string &camera_id, const std::string &channel_id, const FireSignal &signal, FireAlarmLifecycle lifecycle, const std::string &event_id, const std::string &alarm_id, std::uint64_t fire_revision, const std::string &delivery_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- std::string event::EventPayloadBuilder::buildJson(const std::string &camera_id, const std::string &channel_id, const CameraEvent &event, const std::string &snapshot_path, const parking::AppliedParkingRoi *applied_roi=nullptr)
- Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/event/FireAlarmService.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- FireBindingParseResult event::parseFireChannelBindingsStrict(const std::string &specification, const std::string &retained_topic_prefix)
- Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- FireCommandResult event::FireAlarmService::enqueue(Command command) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- FireCommandResult event::FireAlarmService::process(const Command &command) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- FireCommandResult event::FireAlarmService::processAcknowledge(const Command &command) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- FireCommandResult event::FireAlarmService::processSignal(const Command &command, const FireChannelBinding &binding)
- Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- FireCommandResult event::FireAlarmService::submitAcknowledge(std::string channel_id, std::string alarm_id)
- Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- FireCommandResult event::FireAlarmService::submitSignal(FireSignal signal) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- FireStoreMutationResult event::FireAlarmService::applyTransition(const FireStateMutation &mutation)
- Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool event::FireAlarmService::configurationValid() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool event::FireAlarmService::drainCommitted(std::chrono::milliseconds timeout) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool event::FireAlarmService::initialize() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool event::FireAlarmService::start() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool event::FireAlarmService::stop(std::chrono::milliseconds timeout=std::chrono::seconds(30)) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- const FireChannelBinding * event::FireAlarmService::findByChannel(const std::string &channel_id)
- const
- Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- const FireChannelBinding * event::FireAlarmService::findBySensor(const std::string &sensor_id)
- const
- Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- event::FireAlarmService::FireAlarmService(database::EventDatabase &database, Config config, std::vector< FireChannelBinding > bindings, CommitObserver observer={}, AckReadiness ack_readiness={}, MutationBegin mutation_begin={}, MutationEnd mutation_end={})
- Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- event::FireAlarmService::~FireAlarmService() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::size_t event::FireAlarmService::bindingCount() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::string event::FireAlarmService::configurationError() const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- void event::FireAlarmService::closeIngress() noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- void event::FireAlarmService::notify(const FireCommandResult &result) noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `void event::FireAlarmService::run()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/event/FireDeliveryCoordinator.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `bool event::FireDeliveryCoordinator::allChannelsReady()` const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireDeliveryCoordinator::beginDomainMutation(const std::string &channel_id)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireDeliveryCoordinator::beginLifecycleRelease(std::uint64_t observed_generation)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireDeliveryCoordinator::drainDeliveries(std::chrono::milliseconds timeout)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireDeliveryCoordinator::enqueueTransportFact(const mqtt::MqttTransportFact &fact)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireDeliveryCoordinator::hasLifecycleInFlight(const std::string &channel_id)` const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireDeliveryCoordinator::hasRetainedInFlight(const std::string &channel_id)` const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireDeliveryCoordinator::isReady(const std::string &channel_id)` const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireDeliveryCoordinator::isRegularEgressGrantCurrent(const RegularEgressGrant &grant)` const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireDeliveryCoordinator::isRevisionSynchronized(const std::string &channel_id, std::uint64_t fire_revision)` const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireDeliveryCoordinator::publish(const FireOutboxRecord &delivery)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireDeliveryCoordinator::retryDeferred(const std::string &channel_id, std::chrono::steady_clock::time_point now)` const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireDeliveryCoordinator::start()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireDeliveryCoordinator::stop()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::FireDeliveryCoordinator::waitUntilReady(std::chrono::milliseconds timeout)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `event::FireDeliveryCoordinator::FireDeliveryCoordinator(database::EventDatabase &database, Config config, TrackedPublisher publisher, EpochFaultHandler fault_handler={})` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `event::FireDeliveryCoordinator::~~FireDeliveryCoordinator()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::chrono::steady_clock::time_point event::FireDeliveryCoordinator::nextWakeAt()` const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< FireDeliveryCoordinator::RegularEgressGrant >`
- `event::FireDeliveryCoordinator::beginRegularEgress()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::clearRetry(const std::string &channel_id)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::completeDomainMutation(const std::string &channel_id)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::completeRegularEgress()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `void event::FireDeliveryCoordinator::disconnectEpoch(mqtt::MqttConnectionEpoch epoch, const std::string &reason)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::endLifecyleRelease()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::expirePubAckDeadline()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::failCloseAllReadinessLocked()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::invalidateRegularEgressLocked()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::notifyOutboxChanged(const std::string &channel_id)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::processFact(const mqtt::MqttTransportFact &fact)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::pump()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::recomputeReadiness(const std::vector< FireOutboxRecord > &retained_rows, std::uint64_t observed_generation)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::recoverActorFailure(const std::string &reason)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::requeueAllInflight(const std::string &reason)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::run()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::scheduleRetry(const std::string &channel_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::FireDeliveryCoordinator::updateIdleState()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/event/FirePersistence.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `const char * event::toString(FireAlarmLifecyle lifecyle)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const char * event::toString(FireDeliverySinkKind kind)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const char * event::toString(FireDeliveryState state)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const char * event::toString(FireProtocolMode mode)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< FireAlarmLifecyle > event::fireAlarmLifecyleFromString(const std::string &value)`
noexcept
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< FireDeliverySinkKind > event::fireDeliverySinkFromString(const std::string &value)`
noexcept
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< FireDeliveryState > event::fireDeliveryStateFromString(const std::string &value)`
noexcept
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< FireProtocolMode > event::fireProtocolModeFromString(const std::string &value)`
noexcept
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/event/IvaEventResolver.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `std::optional< IvaResolvedTarget > event::IvaEventResolver::resolve(const std::string &cameraId, const CameraEvent &cameraEvent, const std::vector< parking::ParkingSlotConfig > &slots, const std::vector< app::IvaAreaConfig > &areas, std::string *error=nullptr)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/event/IvaOccupancyCoordinator.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `IvaCoordinationResult event::IvaOccupancyCoordinator::handle(IvaOccupancySignal signal, std::chrono::steady_clock::time_point now=std::chrono::steady_clock::now())`
— IVA 액션 한 건을 점유 전이 또는 출차 예약으로 반영한다.
- `event::IvaOccupancyCoordinator::IvaOccupancyCoordinator(const std::vector< parking::ParkingSlotConfig > &slots, std::chrono::milliseconds exitConfirmDelay)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< std::chrono::steady_clock::time_point > event::IvaOccupancyCoordinator::nextDeadline() const`
— 가장 빠른 출차 확인 시각을 반환한다.
- `std::vector< IvaOccupancySignal > event::IvaOccupancyCoordinator::takeDue(std::chrono::steady_clock::time_point now)`
— 확인 시각이 지난 출차 신호를 큐에서 제거해 반환한다.

src/event/IvaSlotOccupancyAggregator.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `event::IvaSlotOccupancyAggregator::IvaSlotOccupancyAggregator(const std::vector< parking::ParkingSlotConfig > &slots)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< IvaSlotOccupancyUpdate > event::IvaSlotOccupancyAggregator::update(const std::string &slotId, const std::string &cameraId, const std::string &videoSourceToken, const std::string &ruleName, bool active)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::IvaSlotOccupancyAggregator::observationKey(const std::string &cameraId, const std::string &videoSourceToken, const std::string &ruleName)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/event/SystemEventReporter.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `bool event::SystemEventReporter::persist(const SystemEvent &event) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::SystemEventReporter::running() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::SystemEventReporter::start()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool event::SystemEventReporter::stop() noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `event::SystemEventReporter::SystemEventReporter(Sink sink)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `event::SystemEventReporter::SystemEventReporter(Sink sink, Config config)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `event::SystemEventReporter::~SystemEventReporter()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `std::size_t event::SystemEventReporter::queuedCount() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::SystemEventReporter::deduplicationKey(const SystemEvent &event)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::serializeSystemEvent(const SystemEvent &event)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::systemAlarmKind(const SystemEvent &event)` — Qt 관제에서 사용하는 센서 오류 공통 분류를 반환한다.
- `std::string event::systemAlarmState(const SystemEvent &event)` — 오류와 복구 여부를 Qt 알람 상태 문자열로 변환한다.
- `std::string event::toString(SystemEventCode code)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::toString(SystemEventSeverity severity)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string event::toString(SystemEventSource source)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::SystemEventReporter::clearRecoveredStateLocked(const SystemEvent &event)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::SystemEventReporter::enqueueLocked(SystemEvent event)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::SystemEventReporter::report(SystemEvent event) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void event::SystemEventReporter::run() noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/http

src/http/ParkingHttpServer.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `HttpServerState http::ParkingHttpServer::state() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool http::ParkingHttpServer::start()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`bool http::ParkingHttpServer::stop(std::chrono::milliseconds timeout=std::chrono::seconds(30))`
- `noexcept`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool http::ParkingHttpServer::usesTls() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
`http::ParkingHttpServer::ParkingHttpServer(database::EventDatabase &database, ServerConfig config, settings::OverstayThresholdService *overstay_settings=nullptr, settings::ParkingRoiSettingsService *roi_settings=nullptr)`
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `http::ParkingHttpServer::~ParkingHttpServer()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void http::ParkingHttpServer::closeIngress()` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void http::ParkingHttpServer::closeIngressLocked()` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void http::ParkingHttpServer::registerRoutes()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void http::ParkingHttpServer::stopListenerLocked()` `noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src

src/main.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/mqtt

src/mqtt/MosquittoTransport.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `std::unique_ptr< IMqttTransport > mqtt::makeMosquittoTransport()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/mqtt/MqttEndpoint.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `MqttEndpointState mqtt::MqttEndpoint::state() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `MqttPublishResult mqtt::MqttEndpoint::publish(const std::string &topic, const std::string &payload, int qos, bool retain)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `MqttPublishResult mqtt::MqttEndpoint::publishInEpoch(const std::string &topic, const std::string &payload, int qos, bool retain, MqttConnectionEpoch expectedEpoch)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `MqttTrackedPublishResult mqtt::MqttEndpoint::publishTracked(const std::string &topic, const std::string &payload, bool retain, MqttPublishCorrelation correlation, MqttConnectionEpoch expectedEpoch)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::abortActiveEpoch(MqttConnectionEpoch expectedEpoch) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::bindApplicationHandler(ApplicationHandler handler)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::bindTransportObservers(TransportObservers observers)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::quiesceIngress(std::chrono::milliseconds timeout=std::chrono::seconds(30)) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::quiesceIngressLocked(std::chrono::milliseconds timeout) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::quiesceTransportObservers(std::chrono::milliseconds timeout=std::chrono::seconds(30)) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::quiesceTransportObserversLocked(std::chrono::milliseconds timeout) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::start(const MqttConnectionOptions &options, const std::vector< MqttSubscription > &subscriptions)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::stop(std::chrono::milliseconds timeout=std::chrono::seconds(30)) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEndpoint::stopLocked(std::chrono::milliseconds timeout) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `mqtt::MqttEndpoint::MqttEndpoint(std::unique_ptr< IMqttTransport > transport)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `mqtt::MqttEndpoint::~MqttEndpoint()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void mqtt::MqttEndpoint::closeIngress()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void mqtt::MqttEndpoint::closeIngressLocked()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `void mqtt::MqttEndpoint::dispatchApplication(const std::string &topic, const std::string &payload)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void mqtt::MqttEndpoint::dispatchConnected()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void mqtt::MqttEndpoint::dispatchDisconnected(int reason)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void mqtt::MqttEndpoint::dispatchPublished(int messageId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void mqtt::MqttEndpoint::dispatchTransportFact(const MqttTransportFact &fact)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

src/mqtt/MqttEventBridge.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `MqttEndpointState mqtt::MqttEventBridge::state() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `MqttTrackedPublishResult mqtt::MqttEventBridge::publishTrackedFire(const std::string &topic, const std::string &payload, bool retain, MqttPublishCorrelation correlation, MqttConnectionEpoch expectedEpoch)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::abortActiveEpoch(MqttConnectionEpoch expectedEpoch) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::bindCameraSnapshotGenerator(CameraSnapshotGenerator generator)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::bindFireAckHandler(FireAckHandler handler)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::bindIvaOccupancyHandler(IvaOccupancyHandler handler)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::bindParkingRoiResolver(RoiResolver resolver)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::bindRegularEgressAdmission(RegularEgressAdmission admission)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::bindSensorMessageHandler(SensorMessageHandler handler)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::bindTransportFactHandler(TransportFactHandler handler)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::publish(const std::string &topic, const std::string &payload, int qos=1, bool retain=false)` — 화재 알림 등 상위 계층의 일반 MQTT 메시지를 발행한다.
- `bool mqtt::MqttEventBridge::publishApplicationEvent(const std::string &topic, const std::string &payload, int qos=1, bool retain=false)` — 카메라 촬영 요청 등 서버 application 메시지를 발행한다.
- `bool mqtt::MqttEventBridge::publishQtEvent(const std::string &topic, const std::string &payload, int qos=1, bool retain=false)` — Qt 관제 클라이언트용 상태.이벤트를 발행한다.
- `bool mqtt::MqttEventBridge::quiesceIngress(std::chrono::milliseconds timeout=std::chrono::seconds(30))` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::quiesceTransportObservers(std::chrono::milliseconds timeout=std::chrono::seconds(30))` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool mqtt::MqttEventBridge::start()` — Broker 연결, topic 구독과 network loop를 시작한다.
- `bool mqtt::MqttEventBridge::stop(std::chrono::milliseconds timeout=std::chrono::seconds(30))` — Mosquitto loop와 연결을 종료하고 자원을 해제한다.

- ```
mqtt::MqttEventBridge::MqttEventBridge(const app::AppConfig &config, std::vector< std::shared_ptr<
camera::CameraChannel > > &channels, database::EventDatabase &database, snapshot::SnapshotStorage
&snapshot_storage, ocr::OcrWorker &ocr_worker, std::vector< parking::ParkingSlotConfig >
parking_slot_configs, SensorMessageHandler sensor_message_handler={}, FireAckHandler
fire_ack_handler={}, IvaOccupancyHandler iva_occupancy_handler={}, std::unique_ptr< IMqttTransport
> transport=makeMosquittoTransport(), notification::TelegramChannelNotifier
• *telegram_notifier=nullptr)
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• mqtt::MqttEventBridge::~MqttEventBridge() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
std::string mqtt::stableEventIdentity(const std::string &topic, const std::string &payload, const
• std::string &discriminator)
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• void mqtt::MqttEventBridge::closeIngress() noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• void mqtt::MqttEventBridge::onMessage(const std::string &topic, const std::string &payload) — 수신 메
시지를 정규화하고 이벤트별 처리 흐름을 실행한다.
• void mqtt::MqttEventBridge::processCameraEvent(event::CameraEvent camera_event) — 분리된 카메라 이벤트 한
건을 기존 IVA/일반 이벤트 흐름으로 처리한다.
```

## src/notification

### src/notification/TelegramApiClient.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- bool notification::TelegramApiClient::sendMessage(const std::string &text) const — Doxygen 설명이 없어 선 언과 호출부를 함께 확인해야 한다.
- notification::TelegramApiClient::TelegramApiClient(const Config &config) — Doxygen 설명이 없어 선언과 호출부 를 함께 확인해야 한다.

### src/notification/TelegramChannelNotifier.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- bool notification::TelegramChannelNotifier::enqueue(TelegramMessage message) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool notification::TelegramChannelNotifier::running() const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool notification::TelegramChannelNotifier::sendWithRetry(const TelegramMessage &message, const std::string &text) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool notification::TelegramChannelNotifier::start() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- notification::TelegramChannelNotifier::TelegramChannelNotifier(const Config &config, const TelegramApiClient &api\_client) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- notification::TelegramChannelNotifier::~TelegramChannelNotifier() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- void notification::TelegramChannelNotifier::run() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- void notification::TelegramChannelNotifier::stop() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### src/notification/TelegramMessageFormatter.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- std::string notification::TelegramMessageFormatter::format(const TelegramMessage &message) const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `std::string notification::TelegramMessageFormatter::truncate(const std::string &text)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/ocr

### src/ocr/GeminiOcrClient.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `OcrResult ocr::GeminiOcrClient::recognizePlate(const std::string &image_path, const std::string &enhanced_image_path=("")) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `OcrResult ocr::GeminiOcrClient::recognizePlateWithModel(const std::string &model, const std::string &image_path, const std::string &enhanced_image_path) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool ocr::GeminiOcrClient::configured() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const char * ocr::toString(OcrErrorKind kind) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `ocr::GeminiOcrClient::GeminiOcrClient(std::string api_key, std::string model, long connect_timeout_sec, long request_timeout_sec, std::string fallback_model="", HttpTransport transport={})` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### src/ocr/OcrWorker.cpp

이미지 전처리·Gemini OCR·DB 반영 비동기 worker 구현.

- `bool ocr::OcrWorker::enabled() const` — Gemini client가 실제 요청 가능한 상태인지 반환한다.
- `ocr::OcrWorker::OcrWorker(GeminiOcrClient client, database::EventDatabase &database, bool preprocess_enabled, ResultCallback result_callback={})` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `ocr::OcrWorker::~OcrWorker()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void ocr::OcrWorker::cancelSession(int session_id)` — 조기 출차 세션의 대기/진행 OCR 결과가 DB와 타이머에 반영되지 않게 한다.
- `void ocr::OcrWorker::enqueue(int session_id, const std::string &slot_id, const std::string &image_path, const std::string &enhanced_image_path={})` — BestShot 이미지를 기존 session의 OCR 작업으로 등록한다.
- `void ocr::OcrWorker::enqueueHallCapture(const HallCaptureTask &task)` — 30/60초 홀 촬영본을 전용 결과 callback이 있는 OCR 작업으로 등록한다.
- `void ocr::OcrWorker::enqueueScene(const std::string &slot_id, const std::string &image_path, const std::string &enhanced_image_path)` — 세션이 아직 없는 IVA scene을 후보 탐색 OCR 작업으로 등록한다.
- `void ocr::OcrWorker::process(const Task &task, HallCaptureResult &hall_result)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void ocr::OcrWorker::run()` — queue 대기, 전처리, OCR, 정규화와 DB 반영을 반복한다.
- `void ocr::OcrWorker::setHallCaptureCallback(HallCaptureCallback callback)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void ocr::OcrWorker::start()` — OCR worker thread를 시작한다.
- `void ocr::OcrWorker::stop()` — 남은 worker를 깨워 종료하고 join한다.

### src/ocr/PlateImageEnhancer.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `PlatePreprocessResult ocr::preprocessPlateImage(const std::string &original_path, bool detect_candidate)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `std::string ocr::enhanceIvaSceneImage(const std::string &original_path)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string ocr::enhancePlateImage(const std::string &original_path)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/ocr/PlateNormalizer.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `bool ocr::isPlausibleKoreanPlate(const std::string &value)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string ocr::normalizePlateNumber(const std::string &value)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/parking

### src/parking/ActiveParkingSessionIndex.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `std::optional< std::string > parking::ActiveParkingSessionIndex::findActiveSessionId(const std::string &slotId) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t parking::ActiveParkingSessionIndex::size() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::ActiveParkingSessionIndex::apply(const ParkingTransitionResult &transition)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::ActiveParkingSessionIndex::clear()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
`void parking::ActiveParkingSessionIndex::clearActive(const std::string &slotId, const std::string &expectedParkingSessionId={})` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::ActiveParkingSessionIndex::setActive(const std::string &slotId, const std::string &parkingSessionId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### src/parking/CaptureScheduler.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `CaptureRequest parking::CaptureScheduler::buildRequest(const std::string &sessionId, const SessionState &session, const CaptureState &capture) const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `CaptureScheduler::ScheduleReport parking::CaptureScheduler::onTransition(const ParkingTransitionResult &transition)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `DispatchOutcome parking::CaptureScheduler::onDispatchResult(const CaptureRequest &request, bool accepted, std::chrono::steady_clock::time_point now)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::CaptureScheduler::CaptureScheduler(CaptureSchedulerConfig config, CaptureTargetResolver &resolver)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< std::chrono::steady_clock::time_point > parking::CaptureScheduler::nextDeadline() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t parking::CaptureScheduler::trackedSessions() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

```
std::vector< CaptureRequest > parking::CaptureScheduler::due(std::chrono::steady_clock::time_point
• now)
```

— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/parking/CaptureSchedulerRuntime.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

```
parking::CaptureSchedulerRuntime::CaptureSchedulerRuntime(CaptureScheduler &scheduler,
• CapturePublisher publisher)
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• parking::CaptureSchedulerRuntime::~CaptureSchedulerRuntime() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야
한다.
• void parking::CaptureSchedulerRuntime::onTransition(const ParkingTransitionResult &transition) —
Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• void parking::CaptureSchedulerRuntime::run() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• void parking::CaptureSchedulerRuntime::start() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• void parking::CaptureSchedulerRuntime::stop() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
```

## src/parking/EvidenceCaptureWorker.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

```
bool parking::EvidenceCaptureWorker::Later::operator()(const Job &left, const Job &right) const
• noexcept
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• bool parking::EvidenceCaptureWorker::canceled(std::int64_t session_id) const — Doxygen 설명이 없어 선언과
호출부를 함께 확인해야 한다.
• bool parking::EvidenceCaptureWorker::expediteOverstay(std::int64_t session_id) — 타이머가 먼저 만료되면 기
존 초과 증거 작업을 즉시 실행 대상으로 만든다.
• bool parking::EvidenceCaptureWorker::restoreSession(EvidenceCaptureRequest request) — 재시작 시 DB에 없
는 증거만 원래 T0 기준으로 다시 예약한다.
• bool parking::EvidenceCaptureWorker::scheduleSession(EvidenceCaptureRequest request) — 시작 즉시 한 장과
T0+지연 한 장을 세션당 한 번 예약한다.
bool parking::EvidenceCaptureWorker::scheduleSessionImpl(EvidenceCaptureRequest request, bool
• include_start, bool include_overstay, bool restored)
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• bool parking::EvidenceCaptureWorker::start() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• const char * parking::toString(EvidenceReason reason) noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인
해야 한다.
parking::EvidenceCaptureWorker::EvidenceCaptureWorker(snapshot::SnapshotStorage &storage,
database::EventDatabase &database, Config config, Completion completion={}, Capture capture={},
• RoiResolver roi_resolver={})
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• parking::EvidenceCaptureWorker::~EvidenceCaptureWorker() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
• std::size_t parking::EvidenceCaptureWorker::pendingCount() const — Doxygen 설명이 없어 선언과 호출부를 함께 확
인해야 한다.
• std::size_t parking::EvidenceCaptureWorker::updateOverstayDelay(std::chrono::milliseconds delay) —
모든 활성 세션의 초과 증거 deadline을 같은 T0 기준으로 재계산한다.
• void parking::EvidenceCaptureWorker::cancelSession(std::int64_t session_id) — VACANT 세션의 아직 실행되지
않은 작업을 취소한다.
• void parking::EvidenceCaptureWorker::emit(EvidenceCaptureResult result) noexcept — Doxygen 설명이 없어 선
언과 호출부를 함께 확인해야 한다.
• void parking::EvidenceCaptureWorker::process(Job job) noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인
해야 한다.
```

- `void parking::EvidenceCaptureWorker::run()` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::EvidenceCaptureWorker::stop()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/parking/HallCaptureCoordinator.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `CaptureImageResult parking::HallCaptureCoordinator::onCaptureImage(const CapturedImage &image)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
`parking::HallCaptureCoordinator::HallCaptureCoordinator(HallCapturePorts ports, int`
- `maxOcrAttempts=2)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
`std::optional< std::int64_t > parking::HallCaptureCoordinator::parseSessionId(const std::string`
- `&value)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t parking::HallCaptureCoordinator::trackedSessions()` const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::HallCaptureCoordinator::onOcrOutcome(const HallOcrOutcome &outcome)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::HallCaptureCoordinator::onTransition(const ParkingTransitionResult &transition)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/parking/HallCaptureExecutor.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `PlateIlluminator::Scope parking::HallCaptureExecutor::illuminate(const CaptureRequest &request)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool parking::HallCaptureExecutor::execute(const CaptureRequest &request)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
`parking::HallCaptureExecutor::HallCaptureExecutor(std::vector< std::shared_ptr<`  
`camera::CameraChannel > > &channels, snapshot::SnapshotStorage &storage, HallCaptureCoordinator`  
`&coordinator, DraftPublisher draftPublisher, camera::CameraSnapshotApiClient`  
`*snapshotApiClient=nullptr, bool rtspFallback=false, PlateIlluminator *illuminator=nullptr,`
- `RoiResolver roiResolver={})` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/parking/HallOcrPolicy.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `HallOcrPolicy::CloseReport parking::HallOcrPolicy::closeSession(const std::string &sessionId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
`HallOcrPolicy::OcrFold parking::HallOcrPolicy::onOcrResult(const std::string &sessionId,`
- `CaptureStage stage, bool recognized)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
`HallOcrPolicy::StageState & parking::HallOcrPolicy::stageRef(SessionState &session, CaptureStage`
- `stage)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
`ImageAction parking::HallOcrPolicy::onImageArrived(const std::string &sessionId, CaptureStage`
- `stage)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `OcrSessionState parking::HallOcrPolicy::state(const std::string &sessionId)` const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const char * parking::toString(ImageAction action)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `const char * parking::toString(OcrSessionState state) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::HallOcrPolicy::HallOcrPolicy(int maxAttempts=2)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t parking::HallOcrPolicy::trackedSessions() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::HallOcrPolicy::openSession(const std::string &sessionId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### **src/parking/ParkingCorrelation.cpp**

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `const char * parking::toString(BestShotEvidenceKind kind) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### **src/parking/ParkingOccupancyConfirmationGate.cpp**

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `ParkingOccupancyConfirmationGate::Decision parking::ParkingOccupancyConfirmationGate::evaluate(const ParkingSensorEvent &event, bool slotAlreadyOccupied)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::ParkingOccupancyConfirmationGate::ParkingOccupancyConfirmationGate(std::chrono::milliseconds confirmThreshold)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< std::chrono::steady_clock::time_point > parking::ParkingOccupancyConfirmationGate::nextDeadline() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< ParkingSensorEvent > parking::ParkingOccupancyConfirmationGate::takeDue(std::chrono::steady_clock::time_point monotonicNow, std::chrono::system_clock::time_point wallNow)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### **src/parking/ParkingOccupancySession.cpp**

센서 기반 주차 세션의 시간·완료 상태 불변식 구현.

- `ParkingSessionState parking::ParkingOccupancySession::state() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool parking::ParkingOccupancySession::active() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const std::optional< std::chrono::system_clock::time_point > & parking::ParkingOccupancySession::endedAt() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const std::string & parking::ParkingOccupancySession::sensorId() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const std::string & parking::ParkingOccupancySession::sessionId() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const std::string & parking::ParkingOccupancySession::slotId() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::ParkingOccupancySession::ParkingOccupancySession(std::string sessionId, std::string slotId, std::string sensorId, std::chrono::system_clock::time_point startedAt, std::chrono::steady_clock::time_point startedAtMonotonic)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.



- `std::chrono::steady_clock::time_point parking::ParkingOccupancySession::startedAtMonotonic() const`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::chrono::system_clock::time_point parking::ParkingOccupancySession::startedAt() const noexcept`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::ParkingOccupancySession::complete(std::chrono::system_clock::time_point endedAt)` — 활성 세션을 완료하며 시작보다 이른 종료 시각은 허용하지 않는다.

## src/parking/ParkingSensorSequenceGuard.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `bool parking::ParkingSensorSequenceGuard::accept(const ParkingSensorEvent &event, std::string *reason=nullptr)`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::ParkingSensorSequenceGuard::clear()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::ParkingSensorSequenceGuard::reset(const std::string &sensorId)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/parking/ParkingSessionWorker.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `parking::ParkingSessionWorker::ParkingSessionWorker(std::vector< ParkingSlotConfig > slotConfigs,`  
`std::chrono::milliseconds confirmThreshold=std::chrono::milliseconds::zero())`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< ParkingTransitionResult > parking::ParkingSessionWorker::onSensorEvent(const`  
`ParkingSensorEvent &event)`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t parking::ParkingSessionWorker::slotCount() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::ParkingSessionWorker::addSink(TransitionSink sink)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/parking/ParkingSlotConfig.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `std::vector< ParkingSlotConfig > parking::ParkingSlotConfigLoader::loadFromFile(const std::string`  
`&path)`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< ParkingSlotConfig > parking::ParkingSlotConfigLoader::parse(const std::string`  
`&jsonText)`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/parking/ParkingSlotManager.cpp

OCCUPIED/VACANT 이벤트의 주차 세션 상태 전이 구현.

- `ParkingTransitionResult parking::ParkingSlotManager::handle(const ParkingSensorEvent &event)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const ParkingSlot * parking::ParkingSlotManager::findSlot(const std::string &slotId) const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const char * parking::toString(ParkingTransitionCode code) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::ParkingSlotManager::ParkingSlotManager(std::vector< ParkingSlotConfig > configs)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `std::size_t parking::ParkingSlotManager::activeSlotCount() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::size_t parking::ParkingSlotManager::slotCount() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string parking::ParkingSlotManager::createSessionId(const std::string &slotId,`  
`std::chrono::system_clock::time_point startedAt)`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/parking/ParkingTriggerCoordinator.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `BestShotAttachResult parking::ParkingTriggerCoordinator::attachBestShotIfActive(const`  
`CommittedCorrelationLease &lease, BestShotEvidenceKind kind, const std::string &image_ref, const`  
`std::string &image_path, const std::string &plate_text={})`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `ParkingCorrelationMatch parking::ParkingTriggerCoordinator::resolve(const std::string &camera_id,`  
`const std::string &channel_id, const std::string &object_id) const`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::ParkingTriggerCoordinator::ParkingTriggerCoordinator(database::EventDatabase &database,`  
`int correlation_window_ms=8000, Clock clock={})`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::chrono::milliseconds parking::ParkingTriggerCoordinator::correlationWindow() const noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::int64_t parking::ParkingTriggerCoordinator::nowEpochMs() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::int64_t parking::ParkingTriggerCoordinator::systemNowEpochMs()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/parking/PlateIlluminator.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `PlateIlluminator::Scope & parking::PlateIlluminator::Scope::operator=(Scope &&other) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `PlateIlluminator::Scope parking::PlateIlluminator::illuminate(const CaptureRequest &request)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `PlateIlluminator::Scope parking::PlateIlluminator::illuminate(const CaptureRequest &request,`  
`std::chrono::system_clock::time_point now)`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool parking::PlateIlluminator::needsLight(const CaptureRequest &request,`  
`std::chrono::system_clock::time_point now)`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool parking::PlateIlluminator::send(const std::string &sensorId, const char *state, std::uint32_t`  
`sequence)`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool parking::isNightWindow(int hour, int startHour, int endHour) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool parking::isNightWindow(std::chrono::system_clock::time_point now, int startHour, int endHour)`  
`noexcept`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::PlateIlluminator::PlateIlluminator(PlateIlluminatorConfig config, LedCommandSender sender)`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- parking::PlateIlluminator::Scope::Scope(PlateIlluminator &owner, std::string sensorId, std::uint32\_t sequence) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- parking::PlateIlluminator::Scope::Scope(Scope &&other) noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- parking::PlateIlluminator::Scope::~Scope() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- void parking::PlateIlluminator::Scope::release() noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- void parking::PlateIlluminator::forget(const std::string &sessionId) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- void parking::PlateIlluminator::forget(std::int64\_t sessionId) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- void parking::PlateIlluminator::markPlateUnreadable(std::int64\_t sessionId) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- void parking::PlateIlluminator::markResolved(std::int64\_t sessionId) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- void parking::PlateIlluminator::note(std::int64\_t sessionId, SessionOcr state) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/parking/SensorSlotIndex.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- parking::SensorSlotIndex::SensorSlotIndex(const std::vector< ParkingSlotConfig > &configs) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
std::optional< SensorSlotMatch > parking::SensorSlotIndex::findBySensorId(const std::string &sensorId) const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::size\_t parking::SensorSlotIndex::size() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/parking/SlotTransitionActor.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- SlotSubmitResult parking::SlotTransitionActor::submit(const SlotTransitionCommand &command) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotTransitionActor::ingressOpen() const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotTransitionActor::processDueDeadlines(std::int64\_t nowEpochMs) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotTransitionActor::processEffects(std::int64\_t nowEpochMs) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotTransitionActor::processIngress(std::int64\_t nowEpochMs) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotTransitionActor::processRunnable(std::int64\_t nowEpochMs) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotTransitionActor::start() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotTransitionActor::stopAndDrain(std::chrono::milliseconds timeout) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking::SlotTransitionActor::waitUntilIdle(std::chrono::milliseconds timeout) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.  
parking::SlotTransitionActor::SlotTransitionActor(database::SessionTransitionStore &store, Config config, EffectSink effectSink) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- parking::SlotTransitionActor::~SlotTransitionActor() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `std::int64_t parking::SlotTransitionActor::nowEpochMs()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::SlotTransitionActor::closeIngress()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking::SlotTransitionActor::run()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/parking/SlotTransitionTypes.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `bool parking::sameSlotSourceFact(const SlotTransitionCommand &lhs, const SlotTransitionCommand &rhs)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const char * parking::toString(SlotCommandKind value)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `const char * parking::toString(SlotCommandStatus value)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< SlotCommandKind > parking::slotCommandKindFromString(const std::string &value)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< SlotCommandStatus > parking::slotCommandStatusFromString(const std::string &value)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/sensor

### src/sensor/FireSensorMessage.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `const char * sensor::toString(FireSensorState state)` noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### src/sensor/HallParkingService.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `HallParkingWorkQueue::PushResult sensor::HallParkingWorkQueue::push(HallParkingWorkItem item)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool sensor::HallParkingService::applyCommittedEffects(const parking::CommittedOccupancyTransition &transition)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool sensor::HallParkingService::handleCameraIvaSignal(const event::IvaOccupancySignal &signal)` — WiseAI IVA 액션을 기존 세션·정리 흐름으로 연결한다.
- `bool sensor::HallParkingService::handleLine(const std::string &line, const std::string &transport="mqtt-test")` — SENSOR:HALLxx:OCCUPIED/VACANT 메시지 한 줄을 처리한다.
- `bool sensor::HallParkingService::removeEarlyDepartureImages(std::int64_t session_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool sensor::HallParkingService::stop(std::chrono::milliseconds timeout=std::chrono::seconds(30))` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool sensor::HallParkingWorkQueue::canAccept(const parking::ParkingSensorEvent &event)` const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `bool sensor::HallParkingWorkQueue::empty()` const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `parking::SlotTransitionCommand sensor::HallParkingService::makeCameraCommand(const event::IvaOccupancySignal &signal)` const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- parking::SlotTransitionCommand sensor::HallParkingService::makeHallCommand(const parking::ParkingSensorEvent &event, const std::string &raw\_line) const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- sensor::HallParkingService::~~HallParkingService() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- sensor::HallParkingWorkQueue::HallParkingWorkQueue(std::size\_t capacity) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::optional< HallParkingWorkItem > sensor::HallParkingWorkQueue::pop() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::size\_t sensor::HallParkingWorkQueue::capacity() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::size\_t sensor::HallParkingWorkQueue::size() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- void sensor::HallParkingService::report(event::SystemEventCode code, event::SystemEventSeverity severity, const std::string &message, const std::string &transport, const std::string &slot\_id={}) noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/sensor/ParkingSensorEventAdapter.cpp

transport 메시지를 주차 도메인 이벤트로 변환한다.

- sensor::ParkingSensorEventAdapter::ParkingSensorEventAdapter(const parking::SensorSlotIndex &slotIndex) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::optional< parking::ParkingSensorEvent > sensor::ParkingSensorEventAdapter::adapt(const SensorProtocolMessage &message, std::string \*error=nullptr) const — 미등록 sensor\_id면 nullptr와 선택적 오류 문자열을 반환한다.

## src/sensor/SensorProtocolParser.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- bool sensor::SensorProtocolParser::isFireLine(const std::string &line) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::optional< FireSensorMessage > sensor::SensorProtocolParser::parseFire(const std::string &line, std::chrono::system\_clock::time\_point receivedAt, std::string \*error=nullptr) const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::optional< SensorProtocolMessage > sensor::SensorProtocolParser::parse(const std::string &line, std::chrono::system\_clock::time\_point receivedAt, std::string \*error=nullptr) const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/settings

### src/settings/OverstayThresholdService.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- OverstayThresholdStatus settings::OverstayThresholdService::status() const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- ThresholdUpdateResult settings::OverstayThresholdService::update(int seconds) — DB 저장 성공 후에만 메모리와 런타임 타이머에 새 값을 적용한다.
- bool settings::OverstayThresholdService::initialize() — DB 값을 로드하며 없으면 bootstrap 기본값을 영구 저장한다.
- bool settings::OverstayThresholdService::isValid(int seconds) noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- int settings::OverstayThresholdService::thresholdSeconds() const noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- `settings::OverstayThresholdService::OverstayThresholdService(database::EventDatabase &database, int bootstrap_seconds=kDefaultSeconds)`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void settings::OverstayThresholdService::setApplyCallback(ApplyCallback callback)` — 타이머·증거 worker 재 예약 callback을 서버 조립 단계에서 주입한다.

## src/settings/ParkingRoiSettingsService.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `ParkingRoiUpdateResult settings::ParkingRoiSettingsService::update(const std::string &slot_id, snapshot::NormalizedRoi roi)`  
— DB 저장 성공 후에만 실행 중 좌표를 즉시 교체한다.
- `bool settings::ParkingRoiSettingsService::initialize()` — DB 저장값을 불러오고, 없으면 AppConfig 좌표를 초기값으로 저장한다.
- `bool settings::ParkingRoiSettingsService::isValid(const snapshot::NormalizedRoi &roi) noexcept` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `settings::ParkingRoiSettingsService::ParkingRoiSettingsService(database::EventDatabase &database, const std::vector< app::IvaAreaConfig > &bootstrap)`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< parking::AppliedParkingRoi > settings::ParkingRoiSettingsService::parse(const std::string &slot_id, const std::string &value)`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< parking::AppliedParkingRoi >`
- `settings::ParkingRoiSettingsService::resolveForUse(const std::string &slot_id) const`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::optional< snapshot::NormalizedRoi > settings::ParkingRoiSettingsService::roiForSlot(const std::string &slot_id) const`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string settings::ParkingRoiSettingsService::serialize(const parking::AppliedParkingRoi &roi)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string settings::ParkingRoiSettingsService::settingKey(const std::string &slot_id)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::vector< ParkingRoiSetting > settings::ParkingRoiSettingsService::list() const` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/snapshot

### src/snapshot/SnapshotStorage.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `StoredImagePair snapshot::SnapshotStorage::saveCameraApiHallCapture(const std::string &channel_id, std::int64_t session_id, const std::string &slot_id, const std::string &capture_stage, const NormalizedRoi &roi, const std::vector< unsigned char > &original_jpeg, const std::vector< unsigned char > &enhanced_jpeg)`  
— 카메라 CAP original/enhanced JPEG를 같은 슬롯 ROI로 잘라 저장한다.
- `StoredImagePair snapshot::SnapshotStorage::saveCameraApiIvaSnapshot(const std::string &channel_id, const std::string &slot_id, const NormalizedRoi &roi, const std::vector< unsigned char > &original_jpeg, const std::vector< unsigned char > &enhanced_jpeg)`  
— 세션이 없는 IVA 이벤트의 original/enhanced JPEG를 ROI로 잘라 저장한다.
- `cv::Mat snapshot::SnapshotStorage::waitForFullFrame(const std::shared_ptr< camera::CameraChannel > &channel)`  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- snapshot::SnapshotStorage::SnapshotStorage(const std::string &snapshot\_dir, int snapshot\_frame\_wait\_ms, std::atomic< bool > &running)  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::string snapshot::SnapshotStorage::saveAreaSnapshot(const std::shared\_ptr< camera::CameraChannel > &channel, const std::string &slot\_id, const NormalizedRoi &roi, const std::string &filename\_prefix, std::int64\_t session\_id=-1, const std::string &stage\_directory={})  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::string snapshot::SnapshotStorage::saveEvidenceSnapshot(const std::shared\_ptr< camera::CameraChannel > &channel, std::int64\_t session\_id, const std::string &slot\_id, const std::string &evidence\_reason, const NormalizedRoi &roi)  
— 세션·증거 종류가 포함된 이름으로 최신 ROI 원본을 저장한다.
- std::string snapshot::SnapshotStorage::saveFullSizeSnapshot(const std::shared\_ptr< camera::CameraChannel > &channel)  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::string snapshot::SnapshotStorage::saveHallCaptureSnapshot(const std::shared\_ptr< camera::CameraChannel > &channel, std::int64\_t session\_id, const std::string &slot\_id, const std::string &capture\_stage, const NormalizedRoi &roi)  
— 30/60초 홀 촬영본을 실제 DB 세션 ID가 포함된 이름으로 저장한다.
- std::string snapshot::SnapshotStorage::saveIvaAreaSnapshot(const std::shared\_ptr< camera::CameraChannel > &channel, const std::string &slot\_id, const NormalizedRoi &roi)  
— Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## src/timer

### src/timer/EventManager.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- bool parking\_timer::EventManager::publish(std::string\_view event\_type, std::string\_view slot\_id, std::string\_view car\_number, std::string\_view occurred\_at, std::string\_view detail={}, std::int64\_t session\_id=-1)  
— slot\_id, 차량, 시각과 상세 근거를 JSON 이벤트로 출력한다.
- void parking\_timer::EventManager::setPublisher(Publisher publisher) — JSON 로그 외에 MQTT 등 외부 전달 callback을 설정한다.

### src/timer/ParkingSlotManager.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- EntryResult parking\_timer::ParkingSlotManager::handleEntry(const std::string &slot\_id, const std::string &car\_number, const std::string &image\_path\_1={})  
— EV/PHEV 입차만 세션과 타이머로 등록하고 중복 입차를 거부한다.
- EntryResult parking\_timer::ParkingSlotManager::handleRecognizedSession(std::int64\_t session\_id, const std::string &slot\_id, const std::string &car\_number)  
— 카메라 흐름이 이미 만든 세션을 중복 INSERT 없이 타이머에 등록한다.
- bool parking\_timer::ParkingSlotManager::start() noexcept — Starts the owned timer worker after runtime teardown is armed.
- parking\_timer::ParkingSlotManager::ParkingSlotManager(EventDatabase &database, EventManager &events, std::chrono::milliseconds parking\_timeout, TimerManager::EvidenceProvider &evidence\_provider={})  
— 입·출차 상태 전이와 EV 점유 타이머를 조정하는 관리자를 생성한다.
- std::chrono::milliseconds parking\_timer::ParkingSlotManager::parkingTimeout() const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

- std::optional< LogRecord > parking\_timer::ParkingSlotManager::handleCommittedExit(std::int64\_t session\_id, const std::string &slot\_id) — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::optional< LogRecord > parking\_timer::ParkingSlotManager::handleExit(const std::string &slot\_id) — 활성 세션을 출차 처리하며 없으면 nullptr를 반환한다.
- std::size\_t parking\_timer::ParkingSlotManager::pendingTimerCount() const — lazy-canceled 항목을 포함한 현재 우선순위 큐 크기를 반환한다.
- std::size\_t parking\_timer::ParkingSlotManager::restoreActiveSessions() — 서버 재시작 시 DB의 EV/PHEV 활성 세션을 타이머 큐에 복구한다.
- std::size\_t parking\_timer::ParkingSlotManager::updateParkingTimeout(std::chrono::milliseconds parking\_timeout) — 활성 EV/PHEV 세션을 원래 T0 기준 새 제한시간으로 모두 재예약한다.

## src/timer/RuntimeConfig.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- RuntimeConfig parking\_timer::RuntimeConfig::load(const std::filesystem::path &file) — KEY=VALUE 형식의 설정 파일을 읽어 런타임 설정을 만든다.
- void parking\_timer::RuntimeConfig::applyEnvironment() — 지원하는 환경변수로 현재 런타임 설정을 덮어쓴다.

## src/timer/TimerManager.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- bool parking\_timer::TimerManager::LaterDeadline::operator()(const TimerItem &left, const TimerItem &right) const noexcept — priority\_queue에서 더 늦은 항목의 우선순위를 낮추는 비교 연산자.
- bool parking\_timer::TimerManager::isCurrent(const TimerItem &item) const — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- bool parking\_timer::TimerManager::start() noexcept — Starts the deadline worker after the runtime teardown guard exists.
- parking\_timer::TimerManager::TimerManager(EventDatabase &database, ViolationCallback callback, ErrorCallback error\_callback={}, std::mutex \*transition\_mutex=nullptr, EvidenceProvider evidence\_provider={}) — DB와 callback을 연결하되 deadline worker는 아직 시작하지 않는다.
- parking\_timer::TimerManager::~TimerManager() — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- std::size\_t parking\_timer::TimerManager::pendingCount() const — 아직 worker가 소비하지 않은 큐 항목 수를 반환한다.
- void parking\_timer::TimerManager::processExpired(TimerItem item) — 만료 노드를 DB 조건부 UPDATE로 검증하고 위반 callback을 발생시킨다.
- void parking\_timer::TimerManager::reportError(const TimerItem &item, std::string message) noexcept — 타이머 오류를 등록된 callback 또는 표준 오류 출력으로 안전하게 보고한다.
- void parking\_timer::TimerManager::reschedule(std::int64\_t log\_id, std::string slot\_id, std::string car\_number, std::chrono::milliseconds delay) — 기존 세션 deadline을 무효화하고 새 기준시간으로 교체한다.
- void parking\_timer::TimerManager::retryAfterDatabaseError(TimerItem item, std::string message) noexcept — 일시적인 SQLite 오류가 난 타이머를 지수 backoff로 다시 예약한다.
- void parking\_timer::TimerManager::retryAfterEvidencePending(TimerItem item) noexcept — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- void parking\_timer::TimerManager::run() — 가장 이른 deadline만 기다리며 만료 노드를 처리하는 worker 루프.
- void parking\_timer::TimerManager::schedule(std::int64\_t log\_id, std::string slot\_id, std::string car\_number, std::chrono::milliseconds delay) — 불변 session ID의 위반 deadline을 큐에 등록하고 worker를 깨운다.



- `void parking_timer::TimerManager::scheduleImpl(std::int64_t log_id, std::string slot_id, std::string car_number, std::chrono::milliseconds delay)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void parking_timer::TimerManager::stop()` noexcept — Stops and joins the deadline worker. Safe to call repeatedly.

## src/timer/Types.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `const char * parking_timer::toString(const VehicleCategory category)` noexcept — 차량 분류 열거형을 로그와 화면에 사용할 문자열로 변환한다.
- `std::string parking_timer::utcNow()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `std::string parking_timer::utcString(const std::chrono::system_clock::time_point now)` — 현재 시스템 시각을 밀리초 정밀도의 ISO-8601 UTC 문자열로 만든다.

## src/timer/main.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `int main(int argc, char *argv[])` — 설정·SQLite·타이머 관리자를 초기화하고 자동 데모 또는 REPL을 실행한다.

## src/util

### src/util/Logger.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `bool util::logEnabled(const std::string &tag)` — 해당 태그가 현재 로그 설정에서 켜져 있는지 확인한다.
- `void util::logError(const std::string &message)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void util::logInfo(const std::string &message)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void util::logLine(const std::string &level, const std::string &message)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `void util::logWarn(const std::string &message)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### src/util/StringUtil.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `std::string util::jsonEscape(const std::string &input)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### src/util/TimeUtil.cpp

이벤트와 파일명에 사용하는 시각 변환 구현.

- `std::string util::isoString(std::chrono::system_clock::time_point value)` — 지정한 system\_clock 시각을 로컬 ISO-8601 문자열로 변환한다.
- `std::string util::nowIsoString()` — 이벤트 payload와 DB에 사용할 ISO 형식 현재 시각을 반환한다.
- `std::string util::nowStringForFilename()` — 파일명에 안전한 현재 시각 문자열을 반환한다.

### src/util/UrlMasker.cpp

Raspberry Pi 서버의 런타임 구현을 담당한다.

- `std::string util::hideUrlForLog(const std::string &url)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## tests/integration

### tests/integration/CameraIvaOccupancyIntegrationTest.cpp

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main(int argc, char *argv[])` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### tests/integration/CameraSnapshotApiClientTest.cpp

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### tests/integration/DbManagerTest.c

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main(int argc, char **argv)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static int check(const char *name, int result)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### tests/integration/EvidenceCaptureWorkerTest.cpp

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### tests/integration/FireBrokerRestartProcessTest.cpp

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main(int argc, char **argv)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### tests/integration/FireDeliveryIntegrationTest.cpp

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### tests/integration/GeminiOcrTest.cpp

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main(int argc, char **argv)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### tests/integration/HallCapturePipelineTest.cpp

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### tests/integration/HallTimerIntegrationTest.cpp

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main(int argc, char *argv[])` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### tests/integration/HttpApiTest.cpp

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## **tests/integration/MqttBridgeLifecycleTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## **tests/integration/ParkingCorrelationIntegrationTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## **tests/integration/SlotTransitionActorTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## **tests/integration/UartFireListenerTool.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## **tests/timer**

### **tests/timer/timer\_tests.cpp**

모든 타이머 단위/통합 테스트를 실행한다.

- `int main()` — 모든 타이머 단위/통합 테스트를 실행한다.

## **tests/unit**

### **tests/unit/AppConfigTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### **tests/unit/CallbackLifecycleTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### **tests/unit/CameralvaEventTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### **tests/unit/CaptureSchedulerTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### **tests/unit/FireAlarmTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

#### **tests/unit/GeminiOcrClientTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

#### **tests/unit/HallCaptureCoordinatorTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

#### **tests/unit/HallOcrPolicyTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

#### **tests/unit/MqttTrackedTransportTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

#### **tests/unit/ParkingDomainTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main(int argc, char *argv[])` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

#### **tests/unit/ParkingOccupancyConfirmationGateTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

#### **tests/unit/ParkingSessionWorkerTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

#### **tests/unit/PlatelluminatorTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

#### **tests/unit/SensorParkingPipelineTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main(int argc, char *argv[])` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

#### **tests/unit/SystemEventReporterTest.cpp**

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## tests/unit/UartLoRaDriverTest.cpp

자동 테스트와 회귀 검증 시나리오를 구현한다.

- `int main()` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

## tools

### tools/code\_doc\_generator.cpp

개발·운영·문서화를 지원하는 독립 도구다.

- `int main(int argc, char *argv[])` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### tools/parking\_alert\_ctl.c

개발·운영·문서화를 지원하는 독립 도구다.

- `int main(int argc, char **argv)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static const char * device_path(void)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static int get_state(int fd)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static int parse_u32(const char *text, __u32 *value)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static int parse_u64(const char *text, __u64 *value)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static int update_slot(int fd, unsigned long request, __u16 operation, int argc, char **argv, int use_write)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static int watch_states(int fd)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static void print_state(const struct parking_alert_state *state)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.
- `static void print_usage(const char *program)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.

### tools/parking\_link\_tool.cpp

개발·운영·문서화를 지원하는 독립 도구다.

- `int main(int argc, char **argv)` — Doxygen 설명이 없어 선언과 호출부를 함께 확인해야 한다.